

School of Computer Science

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES



UNIVERSITY OF LEEDS

Final Report

An Offline-First GIS Architecture for Reliable Asset Data Capture in Electricity Distribution Networks

Archie Peter Abraham

**Submitted in accordance with the requirements for the degree of
BSc Engineering and Technology**

2025/26

COMP3932 Synoptic Project (EPA)

The candidate confirms that the following have been submitted:

Items	Format	Recipient(s) and Date
<i>Final Report</i>	<i>PDF file</i>	<i>Uploaded to Minerva (27/04/26)</i>
<i>Link to online code repository</i>	<i>https://github.com/ArchieAbr/EPA</i>	<i>Sent to supervisor and assessor (27/04/26)</i>
<i>Offline Functionality Demonstration Video</i>	<i>https://youtu.be/UII64myT24E</i>	<i>Sent to supervisor and assessor (27/04/26)</i>
<i>PWA Walkthrough</i>	<i>https://youtu.be/8Ufddl6mpas</i>	<i>Sent to supervisor and assessor (27/04/26)</i>

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student) Archie Peter Abraham

Summary

An important definition to understand before reading this report is the difference between “redlining” and “blacklining” and how these terms are used. When a Field Technician is given a work order containing a set of design drawings, these are referred to as blacklines. A Field Technician may then build these assets (or build as designed), meaning no change is needed to the design on the work order, and the Field Technician can accept the design – referred to as blacklining. However, these blacklines may not be suitable for real-world use. The Field Technician will then update the drawings in the work order to reflect the “as-built” design. This will be referred to as redlining. These terms were selected because they provide a physical visual distinction in asset status within the developed platform.

Distribution Network Operators (DNOs) depend upon geospatial asset data to effectively manage their critical grid infrastructure. However, the technicians responsible for maintaining that infrastructure may find themselves without an active network connection while in the field. Geographic Information Systems (GIS) software applications that DNOs utilise are not always designed to operate fully offline; they may lose functionality during a network outage. As a result, these technicians may be forced to rely on outdated and potentially erroneous paper records or workarounds to document the state of the infrastructure they manage.

A solution to this problem is to implement a web application that can function entirely offline, without requiring a connection to the backend server. The backend server will be structured in a way to support offline asset data capture. This report describes the development of a lightweight proof-of-concept Progressive Web Application using technologies such as Flask, Service Workers, IndexedDB local caching, and GeoJSON to represent asset geometry. This report also describes the development of an Action Queue system that will enable the application to continue operating without data loss during network failures. This offline-first architecture allows field technicians to maintain full access to system functionality regardless of network status, enabling them to complete in-field red and black lining in low- or no-connectivity environments.

Testing of the backend and integration of the software components has shown that the software prevents data loss, validates the functionality and performance of the Action Queue, which is drained upon reconnection. This application aims to address the fragility of field data collection. Additionally, it provides a durable, vendor-independent architecture for DNOs without the need for legacy mobile applications.

Acknowledgements

Firstly, I would like to thank my friends and family, who have supported me from the inception to the completion of this dissertation and throughout my university studies.

I would also like to thank my colleagues at PwC, who, in no small part, motivated the selection of this project. I am particularly grateful for the time they devoted to teaching me the essential business process and analysis skills required for the practical application of this research, as well as for synthesising software requirements. Furthermore, I appreciate their early encouragement and enthusiasm for the project's concept, which gave me the confidence to pursue this topic.

Finally, I would like to offer my sincere thanks to Dr David Head for his advice, support and guidance during the process of completing this project. His constant evaluation and thoughtful input into my approach and the final solution have been invaluable. I would also like to thank my assessor, Dr Yongxing Wang, for his constructive feedback on this project's structure and methodology.

Glossary

- **Action Queue:** A durable, ordered local database queue that sequentially stores user operations for deferred server synchronisation.
- **API (Application Programming Interface):** A set of rules allowing different software applications and backend services to communicate.
- **As-built records / As-built design:** Geospatial records reflecting the actual, physical state and location of installed network assets.
- **BDD (Behaviour-Driven Development):** A software engineering approach formalising requirements into testable 'Given/When/Then' scenarios.
- **Blacklining:** The field process of digitally accepting or confirming planned design assets without modification (building-as-designed).
- **CDN (Content Delivery Network):** A geographically distributed network of servers used to efficiently deliver web assets.
- **CNAIM (Common Network Asset Indices Methodology):** A UK industry-standard framework for validating and capturing network asset condition data.
- **CORS (Cross-Origin Resource Sharing):** A security mechanism allowing a web browser to request resources from a different domain than the one serving the web page.
- **CRDTs (Conflict-free Replicated Data Types):** Data structures ensuring consistency across disconnected systems by embedding conflict-resolution rules without a central coordinator.
- **CRUD (Create, Read, Update, Delete):** The four fundamental operations for managing persistent data within a database.
- **DNO (Distribution Network Operator):** Organisations responsible for operating and maintaining the electricity grid infrastructure.
- **EAM (Enterprise Asset Management):** Large-scale software systems used by organisations to manage physical assets and work orders.
- **ETL (Extract, Transform, Load):** Data integration processes used to merge disconnected field edits into a centralised database.
- **GeoJSON:** An open standard format for representing spatial and geographical vector data natively within web browsers.
- **GIS (Geographic Information System):** Software for modelling, viewing, and managing spatial and geographical data.
- **HTTP (Hypertext Transfer Protocol):** Foundational application-layer protocol designed for transferring data like HTML between web browsers (clients) and servers.

- **Idempotency Key:** A unique identifier preventing duplicate server records if a synchronisation attempt is interrupted or retried.
- **IndexedDB:** A standard client-side database built into web browsers for securely storing structured data.
- **JSON (JavaScript Object Notation):** A lightweight, human-readable data-interchange format.
- **LWW (Last-Write-Wins):** A conflict resolution strategy where the most recent synchronised update overwrites previous states.
- **MoSCoW:** A project management framework categorising requirements into "Must have", "Should have", "Could have", and "Won't have".
- **NFR (Non-Functional Requirement):** Engineering constraints of a system (e.g., performance limits, offline durability) rather than specific user features.
- **NoSQL:** A class of database management systems that do not use a traditional relational tabular structure.
- **ORM (Object-Relational Mapper):** A backend programming layer mapping database tables to code objects.
- **OT (Operational Transformation):** A complex, centralised algorithm for resolving data conflicts in real-time collaborative environments.
- **PWA (Progressive Web App):** A web application using modern standards to combine standard website accessibility with the offline capabilities of a native app.
- **QoL (Quality of Life):** Non-essential but beneficial features that improve the overall usability of the software.
- **Redlining:** The process of field engineers spatially modifying planned assets on a map to create accurate "as-built" records.
- **REST (Representational State Transfer):** An architectural style for designing networked applications, using HTTP requests to access and mutate data.
- **Service Worker:** A background browser script that intercepts network requests, enabling offline functionality.
- **SQL (Structured Query Language):** A standard domain-specific language used to manage and manipulate relational databases (e.g., SQLite).
- **Store-and-forward:** A data communication method where information is sent to an intermediate node, stored temporarily, and later forwarded to its destination.
- **UI:** User Interface
- **UX:** User Experience
- **WGS84 (World Geodetic System 1984):** The standard coordinate system and geodetic datum used globally for mapping, navigation, and GPS.

Table of Contents

Summary.....	iii
Acknowledgements.....	iv
Glossary.....	v
Table of Contents	vii
Chapter 1 Introduction and Background Research	1
1.1 Introduction	1
1.2 Aims and Objectives	1
1.2.1 Objectives	2
1.3 Background Research	2
1.3.1 The Connectivity Challenge in Distribution Networks.....	2
1.3.2 Limitations of Enterprise GIS Solutions	3
1.3.3 Offline-First Architectures and Conflict Resolution	4
1.4 User Persona Development.....	7
1.4.1 Primary Stakeholder: The Field Technician	7
1.4.2 Secondary Stakeholder: The Back Office GIS Data Manager.....	7
1.4.3 Synthesis and Impact on System Design	8
1.5 Requirements Refinement	8
1.5.1 Functional Requirements and Prioritisation (MoSCoW).....	8
1.5.2 Non-Functional Requirements (NFRs)	9
1.6 Acceptance Criteria.....	10
1.6.1 Scenario A: Offline Asset Capture and Queuing.....	10
1.6.2 Scenario B: Deterministic Synchronisation upon Reconnection...	10
1.6.3 Scenario C: Error Handling and Queue Isolation	11
Chapter 2 Methodology.....	12
2.1 Methodology Approach and Project Management.....	12
2.1.1 Iterative Development and Project Management.....	12
2.1.2 Version Control and Configuration Management	12
2.1.3 Engineering Standards and Quality Assurance	13
2.2 Technology Stack Evaluation and Justification.....	13
2.2.1 Evaluation of Suitable Technology Choices.....	13
2.2.2 Justification of the Selected Technology Stack	14
2.3 System Architecture Design.....	14
2.3.1 Data Storage and Schema Design	16

Backend Relational Model	16
Frontend Local Storage	17
2.3.2 Synchronisation Logic and State Management.....	18
Action Queue Processing	18
Conflict Resolution Strategy	18
Chapter 3 Implementation and Validation	20
3.1 Overview.....	20
3.2 Tools and Technologies.....	20
3.3 Architectural Evolution and Prototyping	21
3.4 Core System Implementation.....	22
3.4.1 Client-Side State and the Action Queue	22
3.4.2 The Synchronisation Engine.....	23
3.4.3 Backend API and Relational Integrity	24
3.5 System Validation and Testing.....	25
3.5.1 Backend API Validation	25
3.5.2 Frontend Integration Testing	25
3.5.3 Manual Offline Workflow Simulation	25
Chapter 4 Results, Evaluation and Discussion	26
4.1 Evaluation Against Acceptance Criteria	26
4.1.1 Scenario A: Offline Asset Capture and Queuing.....	26
4.1.2 Scenario B: Deterministic Synchronisation upon Reconnection...	26
4.1.3 Scenario C: Error Handling and Queue Isolation	26
4.2 Objectives and Requirements Review	27
4.2.1 Achievement of Project Outcomes	27
4.2.2 Non-Functional Requirements (NFR) Validation (Demo Video Tests)	27
4.2.3 Functional Requirements Delivery Assessment.....	28
4.3 Discussion	28
4.3.1 Architectural Trade-offs	28
4.3.2 Proof-of-Concept Scope and Limitations	28
4.4 Conclusion	29
4.5 Future Work.....	29
4.5.1 Multi-User Concurrent Conflict Resolution.....	29
4.5.2 Enterprise Integration and ETL Pipelines	30
4.5.3 Map Tile Caching at Scale.....	30

4.5.4 Native Application Migration	30
4.5.5 Enhancing Drawing Capabilities and Asset Taxonomy	30
List of References	31
Appendix A Self-appraisal	36
A.1 Critical self-evaluation	36
A.2 Personal reflection and lessons learned	36
A.3 Legal, social, ethical and professional issues	37
A.3.1 Legal issues.....	37
A.3.2 Social issues.....	37
A.3.3 Ethical issues.....	38
A.3.4 Professional issues	38
Appendix B External Materials	39
B.1 Frontend Libraries	39
B.2 Backend Libraries.....	39
B.3 External Data & Tile Service	40
B.4 Browser / Platform APIs	40
B.5 Development Tools	41
B.6 Generative AI Acknowledgement.....	41
Appendix C Supplementary Figures	42
C.1 Supplementary Figures UI.....	42
C.2 Supplementary Figures: Browser Metrics	49
C.3 Test Results	50
C.4 Technical and Repository Supplementary Information.....	52

Chapter 1

Introduction and Background Research

1.1 Introduction

Modern Electricity Distribution Networks, or DNOs, rely on accurate, up-to-date records of their electrical assets to perform tasks such as grid upgrades, asset planning, maintenance, risk management, and restoration of electrical supply after grid faults. For many utilities, this information is difficult to obtain due to the many different information systems used within their organisation. This problem is also reflected within current regulations for DNOs. The regulatory body Ofgem states that DNOs are “aware of less than half of all CERs (Consumer Energy Resources) and distributed energy resources (DERs) on their networks” due to the “limitations to asset data collection”. Furthermore, Ofgem explains that this lack of visibility of asset data “undermines network and system planning,” leading to “[increased] costs for both network companies and consumers” (Ofgem et al., 2025).

For instance, information systems may include systems for managing assets (Enterprise Asset Management (EAM) Software, such as IFS and IBM Maximo), systems for modelling grids (such as Esri’s ArcGIS and General Electric’s Smallworld Electric Office), and work management systems to manage their field crews (such as Oracle Field Service).

While these information systems are often very robust and well-developed for the companies that utilise them, each is inherently challenged by the need to connect to the internet to collect information on those assets. Thus, if any of these systems should fail outside the office in low-connectivity environments, asset data for those systems must be collected another way (such as through paper workarounds), which may introduce the potential for error and delay.

1.2 Aims and Objectives

This project aims to build a proof-of-concept application that allows field engineers to create and edit a limited set of network assets offline and sync those changes with a backend service when online. Only a small set of network assets will be represented within the software, as the goal is to explore the various aspects of building such an application rather than to produce an application that encompasses the numerous features of existing engineering tools.

1.2.1 Objectives

Objective 1

Analyse the connectivity issues and data flows of DNOs to determine the reasons for potentially inaccurate asset data capture and the limitations and strengths of offline capability within existing GIS software packages.

Objective 2

Translate the constraints on technicians and GIS data managers into personas for the proof-of-concept development and define the requirements for each persona.

Objective 3

Build a PWA that allows engineers to access their work orders, design assets, and map tiles without interruption, even when they are completely offline.

Objective 4

Develop an ordered, durable “Action Queue” in IndexedDB that serialises CREATE, UPDATE, DELETE, and ACCEPT operations as idempotent actions.

Objective 5

Build and design asset data-capture forms for technicians to directly record asset condition and design metrics in the field, to promote data completeness at the point of inception.

Objective 6 — Evaluate System Resilience and Usability

Evaluate the prototype against a set of acceptance criteria by testing under simulated network conditions, using scripted tests and video demonstrations.

1.3 Background Research

1.3.1 The Connectivity Challenge in Distribution Networks

Electricity Distribution Networks are geographically extensive and can include both urban and remote rural electrical grids (Damilola, 2013). Field technicians routinely inspect electrical distribution networks and often rely on mobile GIS technologies to document them. However, many electrical distribution networks lack cellular network coverage in remote areas (Chapungo and Postolache, 2025; Gao and Mai, 2018).

Data transfer between mobile devices and mobile networks is often limited by both capacity and cost. Data transferred often will include large vector or raster datasets, which require a strong network connection to transfer successfully (Ying, 2013). When the mobile applications

for recording inspections of electrical distribution network assets fail due to poor network connectivity, technicians may resort to paper forms and ungeofenced photographs of the assets they are working on. Manually entered data is often subject to errors when rekeyed into back-office systems (Grodén, 2025; Fulcrum, 2026).

The breakdown of digital data captured at the field level can degrade the quality of the central asset register, including status monitoring, predictive maintenance and regulatory compliance, with 1-in-3 DNO executives citing asset data quality as a major challenge (eSmart Systems, 2024). The utility industry reports that the quality of asset data has a major impact on trust in digital asset models and on DNO operations (eSmart Systems, 2024; Frank et al., 2025). For instance:

- Errors in data captured in the field will translate to incorrect data within the GIS models of the assets, leading to incorrect asset planning (Damilola, 2013; Wan, 2016).
- The efficiency of a DNO's ability to repair faults in electrical grids is reliant upon the accuracy of the records of those assets; inaccurate records will require companies to spend more time reacting to faults rather than anticipating them. (Roberts, 2025).
- Regulators require utility records to show plans for mitigating risks to electrical grids and customers. If these are not updated, utilities might be noncompliant, and records may be crucial for investigating outages (Frank et al., 2025).

Asset data captured in the field, therefore, appears to be critical to the management of electrical grids; yet connectivity issues and constraints on such data are major barriers to ensuring its timeliness, reliability and pose a financial or regulatory burden.

1.3.2 Limitations of Enterprise GIS Solutions

Vendor offerings in the market today, such as Esri's ArcGIS, provide DNOs with comprehensive geospatial asset management and design tools (Clifford and Curry, 2021). However, the field-capture components of these platforms are primarily designed as extensions of their vendors' centralised systems (Esri, 2026; 1Spatial, 2021). The platform clients are heavyweight native mobile clients tightly coupled to proprietary back-end schemas and licensing models (Halounová, 2007; Sanderson, 2025).

Additionally, these platforms tend to assume that field technicians will be connected to a network for most of their shifts, adopting an "occasionally connected" paradigm rather than a true offline-first architecture. For instance, ArcGIS Field Maps requires that web maps be preconfigured for offline use; while administrators can define map areas in advance, technicians may also download ad hoc areas, though this requires prior administrative configuration of the map (Clifford and Curry, 2021). Esri's own guidance warns that large offline areas can severely degrade synchronisation performance and complicate deployments

(Smulders, 2022; Shaner, Curry, & Pierce, n.d.). Similarly, Oracle Mobile Field Service permits technicians to work offline by caching their changes via a "Store and Forward" mechanism; however, a device cannot access new work or map areas without re-establishing a network connection (Raju and Oracle, 2026). Thus, the functionality available to technicians offline is limited, and any functionality provided is considered degraded compared to a connected state.

Furthermore, the methods by which these platforms reconcile changes between the field technician's device and a centralised server are generally not configurable by the DNO and are vendor-controlled. For instance, while Oracle Mobile Field Service permits developers to create custom plugins for their mobile application through their Mobile Plugin Framework, the capabilities of that framework are limited to user interfaces (UI) for the technician and the back-end server rather than exposing general-purpose local database abstractions for domain-specific conflict management, for direct control by the DNO (Sinha and Oracle, 2026; Raju and Oracle, 2026). Furthermore, the default method by which Esri reconciles changes between the technician's device and the ArcGIS server is based on a "last sync wins" model at the row- or feature-level (Muresan, 2024), though manual resolution is also supported. Additionally, the manufacturer's suggested method for GE Smallworld to resolve conflicts between changes on the technician's device and those of the Smallworld server involves the use of complex ETL (extract, transform, and load) processes outside of Smallworld, such as using another desktop application outside of Smallworld (Globema, 2019; Sanderson, 2025). Each of these implementations indicates that either the platforms have limited functionality when the technician is offline, or that they introduce complexities and bottlenecks for the DNO. Moreover, because these conflicts are abstracted away from the DNO, the company has limited means to ensure data consistency in such scenarios (Kleppmann et al., 2017). Furthermore, these simple or manual methods risk either the loss of vital information updated in the field by technicians or the creation of bottlenecks in geospatial domains for the DNO's operations (Nguyen et al., 2025). Thus, there is a need for a geospatial platform for DNOs whose field technicians will work offline and whose synchronisation protocols do not rely on heavyweight vendors. Such a system could be built on standard web-based technologies that expose the synchronisation protocols between the technician's device and the DNO's server, enabling greater operational control.

1.3.3 Offline-First Architectures and Conflict Resolution

Unlike occasionally connected models, offline-first models are implemented with the assumption that mobile devices will be largely disconnected from a network (Grodén, 2025). The modern web platform provides the tools necessary to implement such models without the need for native applications. For example, Progressive Web Apps (PWAs) can be developed for mobile and desktop devices that would otherwise require heavyweight native application

development, yet provide the same functionality as native applications (Adobe Experience Cloud Team, 2023).

Progressive Web Apps and Service Workers

PWAs utilise Service Workers to intercept network requests, allowing an application to operate autonomously without a network connection, while client-side storage APIs such as IndexedDB persist structured domain data locally (Firtman et al., 2021; Sirigiri et al., 2023). Such an architecture would allow technicians to interact with asset data and save any changes made to that data on their devices, where it persists in a cached state, ready to be synced upon reconnection.

The specifications for Service Workers include various caching strategies, each balancing data freshness and offline resilience (Archibald, 2014). For instance, the cache-first strategy will always use data stored in the application's cache rather than retrieving it from the network, maximising offline use but risking stale data. A similar strategy is stale-while-revalidate, in which data is retrieved from the cache for offline use and revalidated from the network when the application launches (Archibald, 2014; MDN, 2025). Having analysed the literature and use cases of service workers, for the context of a DNO, a hybrid caching strategy is preferred; static assets and map tiles use cache-first or stale-while-revalidate for maximum offline availability, while transactional API calls bypass the cache entirely to preserve data integrity (Archibald, 2014; MDN, 2025).

This strategy has proven successful in other models with similar connectivity requirements. For instance, a Service Workers-based PWA has been implemented in the health application mHealth to allow health workers in Sub-Saharan Africa to input data forms into their mobile devices while they are away from connectivity, and that data automatically syncs with the system backend when they regain access to a network connection (Sirigiri et al., 2023; Kenny et al., 2020). Similarly, platforms like ODK and KoBoToolbox have used these strategies to enable field researchers in areas with no communication infrastructure to enter forms on their mobile devices, which are synced with central databases upon reconnecting to the network (Hartung et al., 2010).

This approach is based on the local-first software principle proposed by Kleppmann et al. in 2019. The model posits that the information on the client device is the single source of truth for the system's data, and any changes made on the device are synchronised with the server, unlike cloud-centric architectures, where the server is the main authority. Applying this concept to DNO fieldwork, this strategy addresses some of the issues we have discussed, namely, maintaining full system usability when offline and not treating network partitions as a degraded state.

Conflict Resolution

As these models allow any number of users to disconnect from the network while still contributing to the system's shared data, some form of conflict resolution must be established. In general, conflict resolution strategies for distributed systems use some form of timestamping to determine which version of the shared data should be used: the record most recently written to. Last-Write-Wins (LWW) is a simplistic and popular choice; it can be problematic for systems that depend on accurate time representation. Clock skew can produce incorrect ordering, and semantically meaningful updates can be silently overwritten (Tossani, 2018). Alternatives to this strategy include Operational Transformation (OT), wherein an operator-based model resolves conflicts between timestamps, and Conflict-free Replicated Data Types (CRDTs), wherein data types are defined with built-in conflict resolution strategies that avoid the issues of LWW while preserving the ability for multiple users to use a system simultaneously (Stichbury, 2022; McKelvey, 2025).

In the context of this project and its intended scope, combining LWW with a queue data structure to serialise Field Technician data-capture actions as fine-grained, idempotent operations would enable testing of these principles. This “Action Queue” would allow actions to be created as objects and saved to a datastore on a technician’s device (Fertig and Braun, 2018; Distributed System Authority, 2026). These objects can then be sent to the server, in sequence, after the technician connects to a network, using standard CRUD operations. By maintaining explicit lifecycle states, such as pending, synced, or in conflict, the queue structure ensures that network failures and data conflicts are observable to the user rather than hidden system errors, allowing for correction (Android Developers, 2026).

An architectural decision for this proof-of-concept model is to define the scope of the local data replica on the client device. Options for database replicas include a full replica, a partial replica, or a shadow copy (Edwards et al., 1997; Saito and Shapiro, 2005). A full replica includes all the information from a master database. A shadow copy includes only the metadata for the objects in the database, whereas a partial replica can be scoped to include only the information relevant to the application's use case and can dynamically change as needed. Thus, in the context of GIS asset data capture, a partial replica would allow a technician to access the relevant data for their work order while minimising the data stored on the technician's device and reducing the complexity of conflict resolution after reconnection to the network, as well as storage overhead.

Finally, handling spatial asset data within this web-native architecture requires lightweight formatting. Because geodatabases are often very large, formats other than geodatabases must be utilised. JSON provides a lightweight, readable format. Furthermore, the GeoJSON

format proposed by Butler et al. (2016) defines several types of JSON objects and how they are combined to represent data on geographic features, their properties, and their spatial extents (Butler et al., 2016). Like JSON, this format is natively compatible with JavaScript and IndexedDB.

1.4 User Persona Development

To engineer a vigorous set of system requirements without direct access to DNO personnel, a persona-driven approach was used. Drawing directly upon the environmental constraints and industry challenges identified in the literature review, two primary user personas were synthesised to drive the system's functional and non-functional requirements.

1.4.1 Primary Stakeholder: The Field Technician

The Field Technician represents the primary operator of the mobile client. Their core objective is the timely execution of work orders and the accurate geospatial capture of as-built network assets (e.g., poles, transformers, and cables).

This persona is often exposed to environmental challenges. For example, much of the existing literature indicates that technicians must work either in remote rural or otherwise low-connectivity environments (Chapungo and Postolache, 2025; Damilola, 2013). Connectivity interruptions often force this persona to utilise pen-and-paper notes due to the constraints outlined in Section 1.3.1 (Grodén, 2025; Fulcrum, 2026).

Thus, the application developed for this persona must emphasise an offline-first user experience (UX). The application must actively support full data entry, including spatial drawings and asset data capture forms, while offline.

1.4.2 Secondary Stakeholder: The Back Office GIS Data Manager

The Back-Office GIS Data Manager is the stakeholder who utilises the geospatial data collected by the Field Technicians from the electrical grid. Their objective is to maintain an accurate representation of the electrical grid in their database for use in various post-inspection or building activities, such as planning, engineering, and regulatory compliance (Ofgem et al., 2025).

The literature indicates that errors in geospatial data capture can lead to inaccuracies in power grid models constructed within the company's engineering and planning departments (Damilola, 2013; Frank et al., 2025). When field technicians use paper workarounds, this may lead to errors and latency in a DNO's central asset register database, which can affect other EAM systems.

For this stakeholder, the system requirements shift towards meeting data governance requirements. For example, the data will have to be auditable, follow a specific schema, and include a method for resolving conflicts.

1.4.3 Synthesis and Impact on System Design

The analysis of these two personas has revealed a dichotomy within the GIS system architecture. The field technician requires flexibility and autonomy from the mobile application, while the back-office manager requires strict, centralised relational consistency and data auditability.

By identifying this dichotomy, the justification for separating concerns in the architecture becomes clear. The PWA will be optimised for functionality without requiring an internet connection, while the backend will maintain a comprehensive system of records and ensure accurate syncing and updates of data mutations.

1.5 Requirements Refinement

Following the development of the user personas, the project's high-level objectives were translated into specific and testable system requirements. Given that this project is a proof of concept, a structured prioritisation framework was required to manage the scope and keep development focused on resolving the core connectivity constraints.

1.5.1 Functional Requirements and Prioritisation (MoSCoW)

The MoSCoW framework (Must have, Should have, Could have, Won't have) was used to categorise the Functional Requirements (FR) (ProductPlan, 2025).

A prioritisation matrix was developed with these core definitions in mind:

1. **Must Have:** Form the fundamental baseline of the offline-first architecture. Failure to deliver these would render the proof of concept invalid for its primary aims.
2. **Should Have:** Features that significantly enhance the operation of the tool but could be omitted without compromising core functionality.
3. **Could Have:** Represent QoL improvements to be implemented only if time permits
4. **Won't Have:** Defines what will not be included to prevent scope creep and focus on the core problem domain. May also include features that are not technically feasible to implement due to complexity or time constraints.

Must Have	Should Have
1. The client app must load and function completely independent of an active network connection. 2. The system must allow users to view pre-cached geospatial data (work orders, network assets) on an interactive map. 3. The system must allow the creation, modification, and deletion of assets while offline 4. All offline data mutations must be recorded and cached locally. 5. The system must deterministically execute cached actions against server-side storage upon network restoration. 6. The UI should clearly indicate the current network status and the volume of pending actions.	1. The app should include spatial drawing tools that allow the user to connect point assets with linear assets 2. Data capture forms should enforce basic client-side validation. 3. The system should automatically calculate cable length to help improve data accuracy
Could Have	Won't Have
1. The UI could provide explicit visual feedback upon the successful synchronisation of individual items.	1. The system will not integrate directly with live EAM systems. 2. The architecture will not support real-time collaborative editing between multiple field technicians. 3. The system will not integrate with live grid data and connections.

Table 1.1: MoSCoW Prioritisation Matrix.

1.5.2 Non-Functional Requirements (NFRs)

NFR1: The system must guarantee that offline data entry is not subject to data loss. The client-side database (IndexedDB) must persist state securely even if the browser process is terminated or the device is powered off unexpectedly

NFR2: The application must transition smoothly between online and offline states without blocking the main UI thread or causing the map rendering engine to crash.

NFR3: The client-side storage architecture must manage geospatial vector data and raster map tiles without exceeding the standard browser quota limits, keeping the application lightweight and avoiding excessive device memory usage.

NFR4: The backend database schema must maintain a chronological audit trail of all synchronised actions.

1.6 Acceptance Criteria

To demonstrate traceability between the user personas and the final product, the high-priority functional requirements were expressed as Behaviour Driven Development (BDD) scenarios. The use of the “Given/When/Then” syntax will provide a deterministic framework for evaluating scenarios and the system's success.

The following criteria focus specifically on the core architectural challenge: ensuring the durability of the offline functionality and database synchronisation.

1.6.1 Scenario A: Offline Asset Capture and Queuing

Purpose: Validate the ability of the Field Technician persona to continue data entry without a network connection.

- **GIVEN** the mobile client is loaded via the PWA shell and the device network status is explicitly disconnected (offline),
- **WHEN** the Field Technician creates a new network asset (e.g., a pole) and submits the associated asset data capture form,
- **THEN** the spatial and attribute data must be immediately persisted to the local IndexedDB storage,
- **AND** a discrete ‘create’ operation must be appended to the local Action Queue with a unique idempotency key,
- **AND** the UI must visually update to indicate a pending action without blocking the user or displaying a browser-level network error.

1.6.2 Scenario B: Deterministic Synchronisation upon Reconnection

Purpose: Verify the automated reconciliation process required by the Back-Office persona.

- **GIVEN** the device contains unsynchronised operations within the local Action Queue,
- **WHEN** the application detects the restoration of a stable network connection,

- **THEN** the client must sequentially transmit the queued actions to the backend Python API in chronological order,
- **AND** upon receiving a successful HTTP 200/201 acknowledgement from the server for each item, the client must remove the corresponding operations from the local queue,
- **AND** the UI must update the pending action count to zero, confirming a globally synchronised state.

1.6.3 Scenario C: Error Handling and Queue Isolation

Purpose: Ensures the failure of a single offline action does not corrupt the entire synchronisation payload.

- **GIVEN** that the application is actively draining the Action Queue to the backend server,
- **WHEN** the server rejects a specific operation (e.g., due to a schema validation failure or a missing foreign key in the SQLite database),
- **THEN** the backend must safely roll back that specific transaction without crashing,
- **AND** the client must flag that specific action as 'failed' within the local queue,
- **AND** the client must continue to process any remaining, independent actions in the queue, isolating the fault and notifying the user.

Chapter 2

Methodology

2.1 Methodology Approach and Project Management

To ensure the application's delivery, an agile software development lifecycle was used. This section describes the methodologies, version control, and standards used in the application's development.

2.1.1 Iterative Development and Project Management

Given the experimental nature of implementing bespoke offline-first synchronisation logic, a more rigid waterfall methodology was inappropriate for this project. Instead, an Agile development approach was applied (Kavlakoglu and IBM, 2025). The development process was carried out in a series of phases, each managed separately on its own branch within GitHub.

2.1.2 Version Control and Configuration Management

A systematic approach to repository management was maintained throughout the project. The project's directory structure separates frontend components from backend components. Furthermore, to enhance security and reduce the risk of accidental or unnecessary file committal, a *.gitignore* file was used to exclude files such as Python cache files, virtual environment files, and environment variables.

Automated shell scripts were also developed to ensure that the application could be tested in a repeatable environment. The script *run.sh* was placed in the root of the project directory. The script automatically starts the Flask server and enables instantaneous database resets by passing the *--reset* argument to *app.py*. This automated process drops all tables in the database and reseeds it with mock work orders, ensuring the testing environment is consistent each time.

Finally, to avoid spending time setting up and managing a cloud environment, the project was developed and tested locally. This approach was also selected because it provided a controlled environment to test the project's core objective: offline resilience. By manually terminating the local server process, network partitioning could be simulated instantaneously and reliably.

2.1.3 Engineering Standards and Quality Assurance

Adherence to standard good practice in software design was heavily prioritised. The architecture follows a strict separation of concerns, decoupling the PWA presentation layer from the data persistence logic and backend APIs.

Validation mechanisms were embedded directly into the project structure. Automated testing scripts, such as 'system-checks.js' for the frontend application logic and 'test_backend.py' for the Python server, enabled quick validation of critical functionality.

2.2 Technology Stack Evaluation and Justification

Before starting the detailed design and implementation phases, an evaluation of potential technology stacks and frameworks was conducted. This was primarily focused on language selection, which, once made, would leave a set of tools available.

2.2.1 Evaluation of Suitable Technology Choices

JavaScript and Progressive Web Application (PWA) Architecture

Implementing a PWA in JavaScript is the most direct method of the offline-first approach identified in the literature review (1.3.3). The use of Service Workers to cache the application shell and tiles (Archibald, 2014), as well as IndexedDB or wrapper libraries such as Dexie.js for client storage, is central to this approach (Firtman et al., 2021; Sirigiri et al., 2023). Another useful feature of this technology stack is its cross-platform nature, which allows it to be deployed across a wide range of devices without requiring multiple native codebases (Adobe Experience Cloud Team, 2023). In addition, the ecosystem offers highly mature web mapping libraries, such as Leaflet and OpenLayers, which are designed to handle complex vector and raster data (Geoapify, 2019). However, a PWA does not provide a fully native experience, and browser-managed storage quotas on the device can limit map availability to certain size constraints (MDN, 2025). Furthermore, access to low-level hardware sensors is often abstracted or restricted compared to native applications (Kaczmarczyk et al., 2022)

Native Android (Java)

This is a highly representative option for industry practice as evidenced by the current vendor options in Chapter 1. This architecture focuses on using the Room Database as the single source of truth for offline data and on background processing APIs, such as WorkManager, for synchronisation tasks when the app is not open (Android Developers, 2026; Guda, 2025). There is also a strong focus on hardware sensors, with native access to low-level sensor hardware, making it ideally suited for accurately capturing GPS telemetry for spatial asset

evaluations (Kaczmarczyk et al., 2022). However, because native Java development is non-trivial, it takes a lot of time to write adapters and permission handlers rather than focusing on the defined problem space (Trogrlic, 2023).

Python

While Python is less suited to the frontend needs of this architecture, its capabilities remain highly valuable for the system's backend infrastructure. Server-side micro-frameworks such as Flask offer a lightweight and highly effective way to build RESTful APIs for managing synchronisation payloads and interacting with the central relational database (Krebs and Martinez, 2025).

2.2.2 Justification of the Selected Technology Stack

Based on an evaluation of each possible architecture for this application, the chosen technology stack of a PWA that employs a vanilla JavaScript frontend, Leaflet as the GIS platform, and a Python Flask backend with an SQLite database was determined to be the optimal technology stack for this proof of concept.

The reason for this decision was the need to develop an offline-first application without the more involved development processes of a native application. The PWA architecture includes all the necessary tools to build a proof-of-concept, offline-first platform for GIS asset data capture and design. Leaflet was selected as the Geographic Information System (GIS) software for the application due to its focus on mobile-friendly mapping, open-source nature, and shallow learning curve (Agafonkin, 2025). OpenLayers was also considered as the selected GIS, offering a more comprehensive set of GIS tools, but was ultimately rejected due to the project's time span and intended complexity (Geoapify, 2019).

Finally, the selection of this specific technology stack was made possible by the project's scope. The goal is to create a proof of concept for the technology and architecture of an offline-first application, thereby avoiding the creation of a production application that could potentially replace systems like Smallworld or ArcGIS. Thus, using JavaScript and Python enables rapid application development with the necessary technologies to support an offline-first data model.

2.3 System Architecture Design

The solution features a decoupled client-server architecture to satisfy the offline-first requirements of field asset capture (Figure 2.1). The frontend includes an orchestrator and a Leaflet-based map engine. The Service Worker acts as a local proxy for the application. To optimise the application's behaviour when offline and avoid using outdated mapping data, the

Service Worker employs a hybrid caching strategy (Section 1.3.3). Static application files are cached first to ensure they load as quickly as possible. OpenStreetMap tiles are accessed using a stale-while-revalidate caching strategy to ensure that previously visited areas of the map are displayed as soon as the application is used offline, while updated map tiles are retrieved in the background once the device reconnects to a network. All API requests bypass the Service Worker (network-only) so that the orchestrator can explicitly manage the application state.

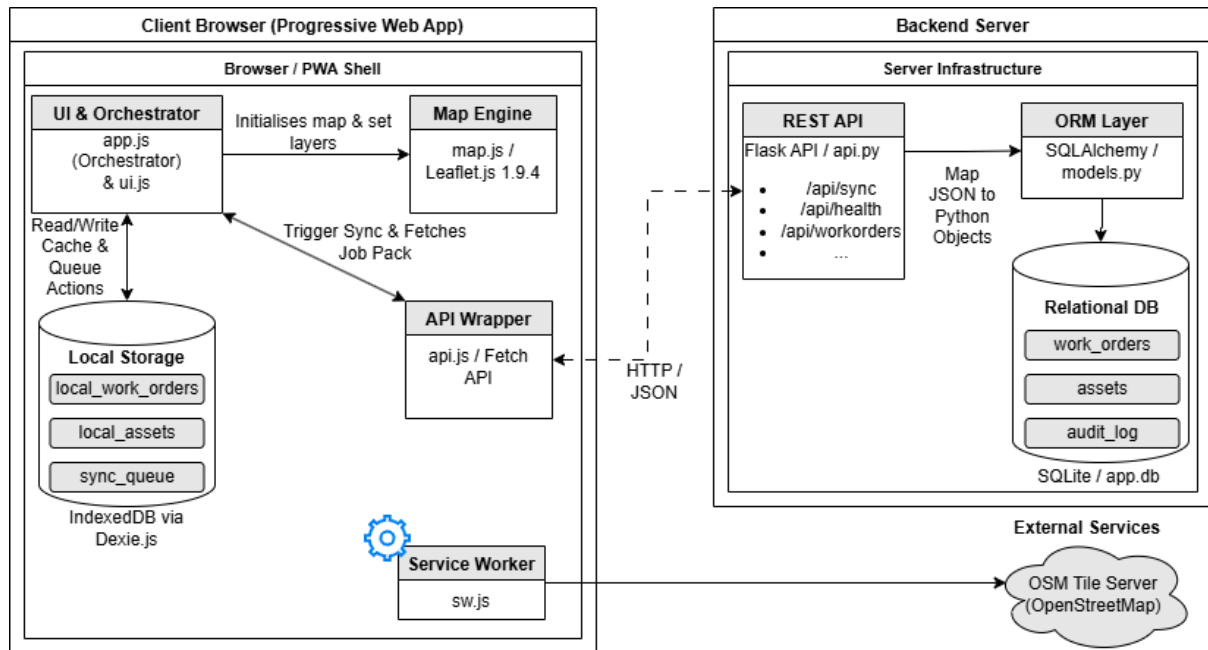


Figure 2.1 High-level system architecture diagram, showing the decoupled interaction between the PWA frontend, local storage mechanisms, and the Python Flask backend.

To support field technicians in clearly distinguishing between planned network topology and captured as-built data, the map layer architecture uses a four-state visual language (Table 2.1).

Layer	Colour	Style	Represents
Planned design (Blacklined)	Black	Dashed	Planned/proposed assets.
Accepted - awaiting sync	Black	Solid	Accepted work order design by the engineer; not yet synced.

New capture / as built - awaiting sync (Redlined)	Red	Solid (dashed if un-synced)	Newly captured as-built data, pending sync.
Existing Server Assets / Registered	Blue	Solid	Assets in the asset register.

Table 2.1: Asset status colour representations.

2.3.1 Data Storage and Schema Design

The architecture employs a dual-database system: one for the central asset register and another for local system replicas.

Backend Relational Model

The decision to use a relational model marked a departure from the initial strategy of storing the data in a GeoJSON file containing the complete asset register. The GeoJSON flat-file approach is simple to implement yet lacks atomicity, concurrency safety, and reliability for a system critical to engineering work. Using a database with data integrity and creating an immutable log of actions performed on the system provides the Action Queue pattern with a robust, reliable foundation. The switch to an SQLite database solved the issues described with the NoSQL approach. Accessed via an SQLAlchemy Object-Relational Mapper (ORM), the database contains three tables: for work orders, assets, and an audit log. The audit log contains fields for the engineer's details, the asset details, and an immutable payload that records the actions performed by the engineer on the asset.

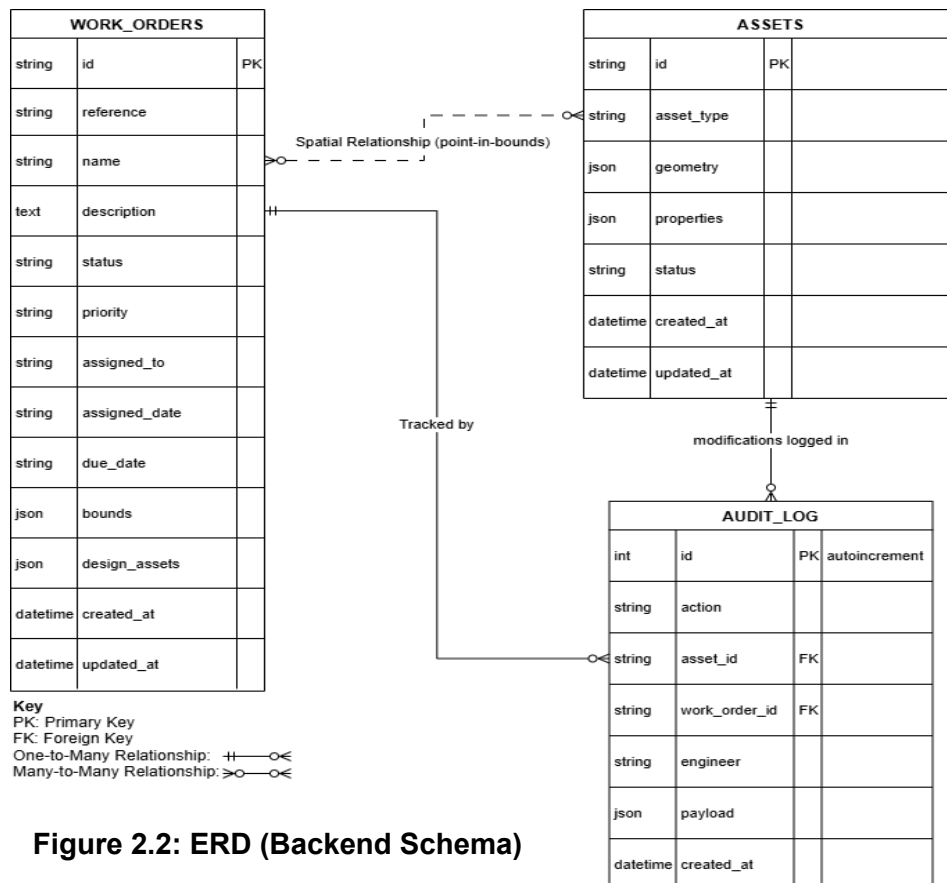


Figure 2.2: ERD (Backend Schema)

Frontend Local Storage

On the client, IndexedDB is used via the Dexie.js wrapper, which replicates the data schema for local storage, as illustrated in Figure 2.3. The frontend schema mirrors the structure of work orders and assets, and includes a queue for synchronising changes to the database (*sync_queue*). This is the introduction of the dedicated Action Queue.

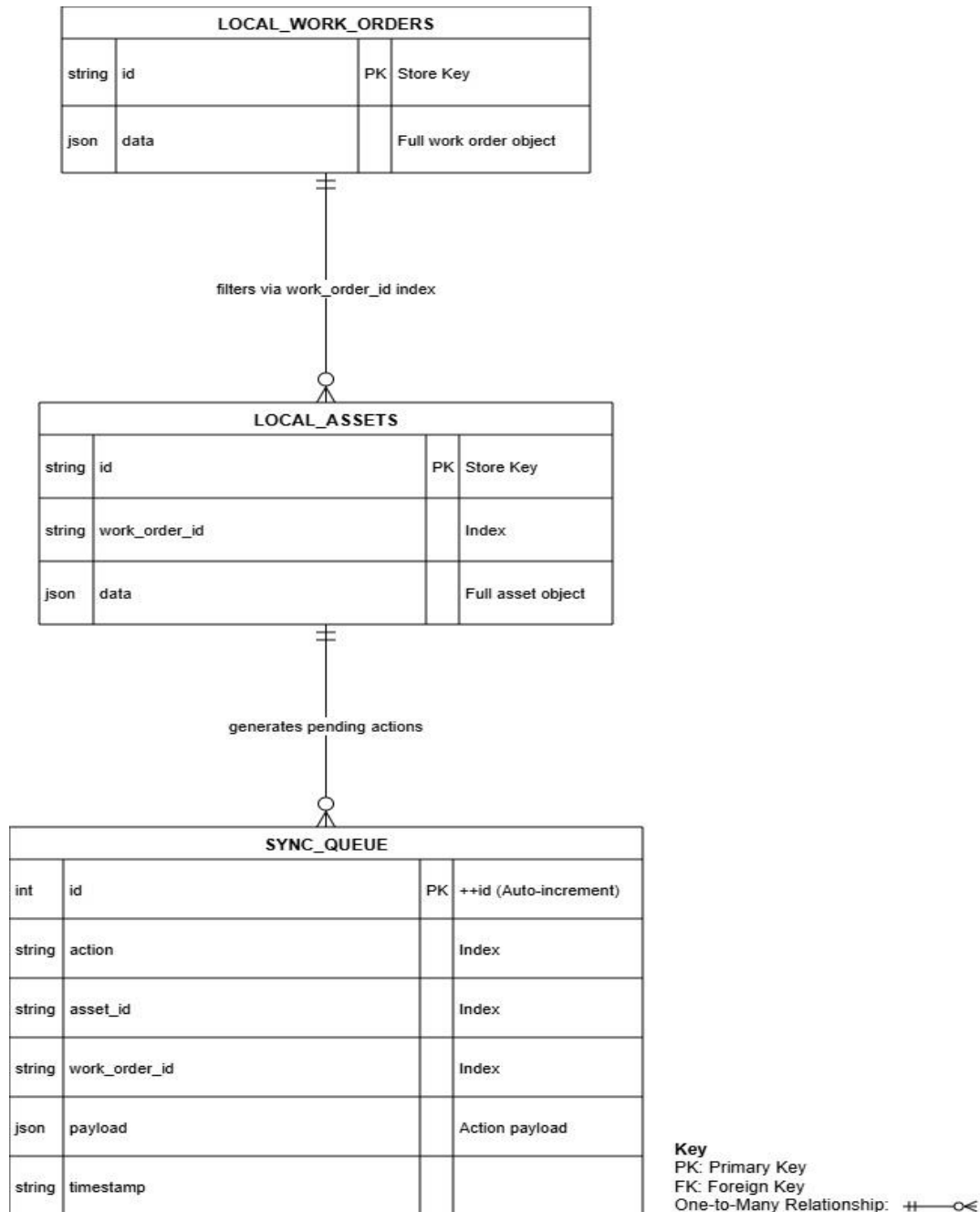


Figure 2.3: Entity-Relationship Diagram showing the frontend NoSQL schema.

Unlike relational database tables, these data stores are NoSQL, which use indexing on fields such as *work_order_id* to quickly find the relevant assets for a work order.

Furthermore, the frontend schema must differentiate between planned 'designed assets' (blacklined) and existing assets. Each work order will have both types of assets cached in the database, allowing the orchestrator to separate planned and installed assets and track newly installed assets separately until they are synchronised with the database.

2.3.2 Synchronisation Logic and State Management

A robust mechanism for managing offline mutations is required to ensure that the system remains reliable in the field.

Action Queue Processing

Rather than directly synchronising static snapshots of local databases, the system records the actions users take within the PWA. When users perform actions while offline, those actions are written into the local cache and then pushed into the *sync_queue*. When the orchestrator detects that the PWA has re-established connectivity to the backend server, the Field Technician can then initiate a sync. During this process, all actions stored in the *sync_queue* are drained chronologically, posting payloads to the backend API, where they are validated, propagated to the central register, and recorded in the audit log before the client is instructed to clear its local queue (see Appendix C Figure C.1.13: UML sequence diagram for more detail).

Connectivity State Handling

To provide feedback to the field engineer regarding the system's connectivity, the orchestrator includes a state machine that monitors the engineer's device connectivity to the backend server (see Figure 2.4). This state machine polls a lightweight health endpoint to determine whether the Field Technician has reestablished a connection. If the application is determined to be disconnected, the UI displays the interface as offline. Should the Field Technician successfully reconnect, the interface updates to reflect an online status.

Conflict Resolution Strategy

Since end-user actions are performed asynchronously relative to the backend servers, a strategy for resolving potential conflicts is required. The conflict resolution strategy chosen is LWW. In this scenario, if the end user performs multiple actions on the same asset while offline, the last action sent to the backend servers will overwrite any previous actions for that asset.

Conflict resolution strategies such as LWW are less complex than strategies such as Operational Transformation (OT) or Conflict-free Replicated Data Types (CRDTs) (see Chapter 1). Given that the application is designed to be a proof of concept for the architecture the complexities of implementing a more sophisticated conflict-resolution strategy are out of

scope for this proof of concept, and is therefore a recognised but necessary trade of due to time constraints and complexity.

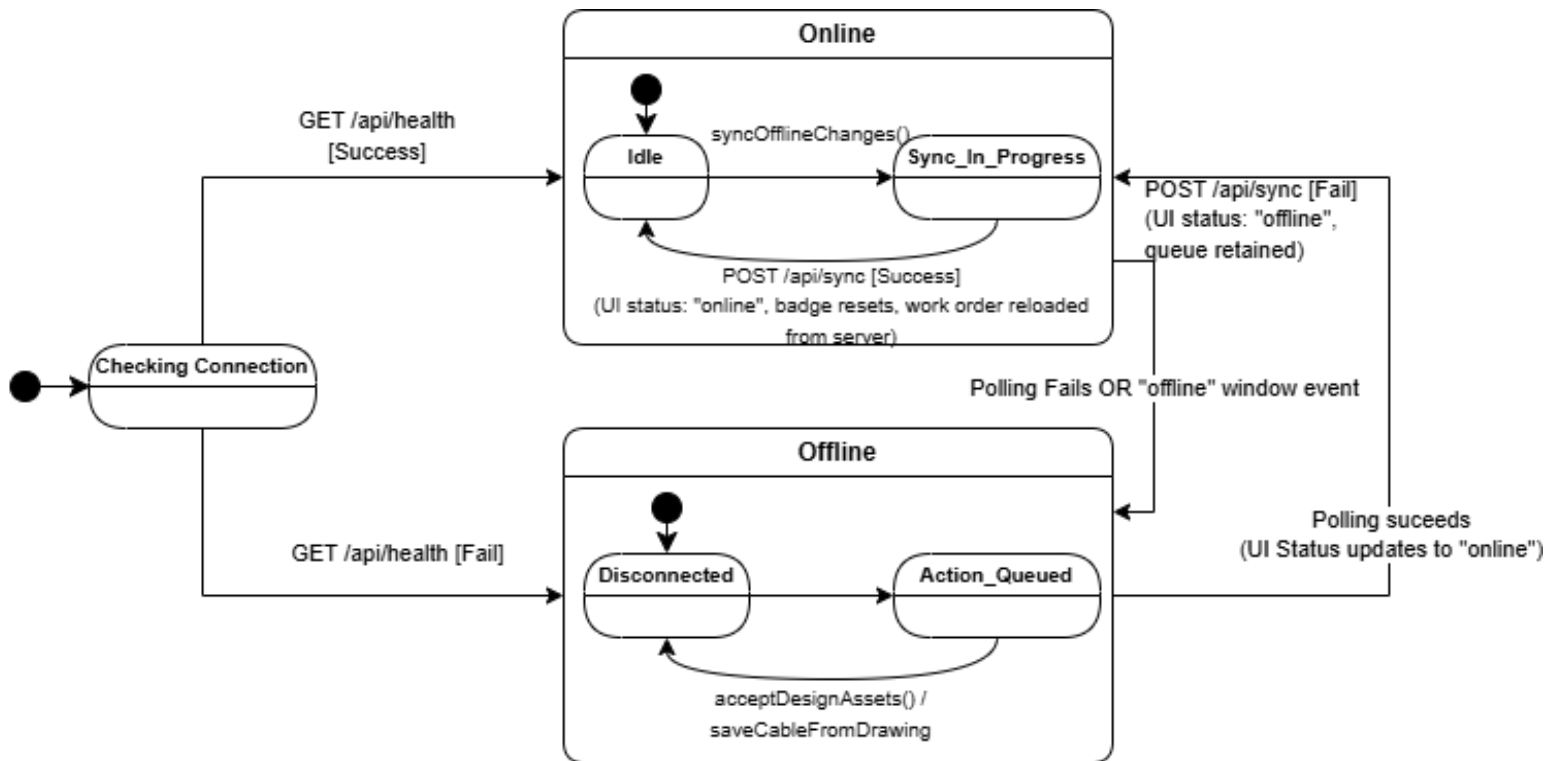


Figure 2.4: UML State Machine Diagram: PWA frontend synchronisation manager.

Symbol	Meaning
● (filled circle)	Initial pseudo-state (entry point)
Rounded rectangle	State
Rounded rectangle with header bar	Composite state (contains sub-states)
Arrow with label	Transition trigger/guard condition
<i>methodName()</i> on transition	Action invoked on that transition
<i>[Condition]</i> in brackets	Guard condition (transition only fires if true)
<i>(UI status: "...")</i> in parentheses	Side effect / internal action note

Table 2.2: Figure Legend for Figure 2.4.

Chapter 3

Implementation and Validation

3.1 Overview

The two-tier architecture described in Chapter 2 was used to create a working proof of concept. The frontend is a vanilla JavaScript PWA served as a set of static files; the backend is a Python Flask API backed by an SQLite database. Figure 3.1 provides a high-level component view of the working system.

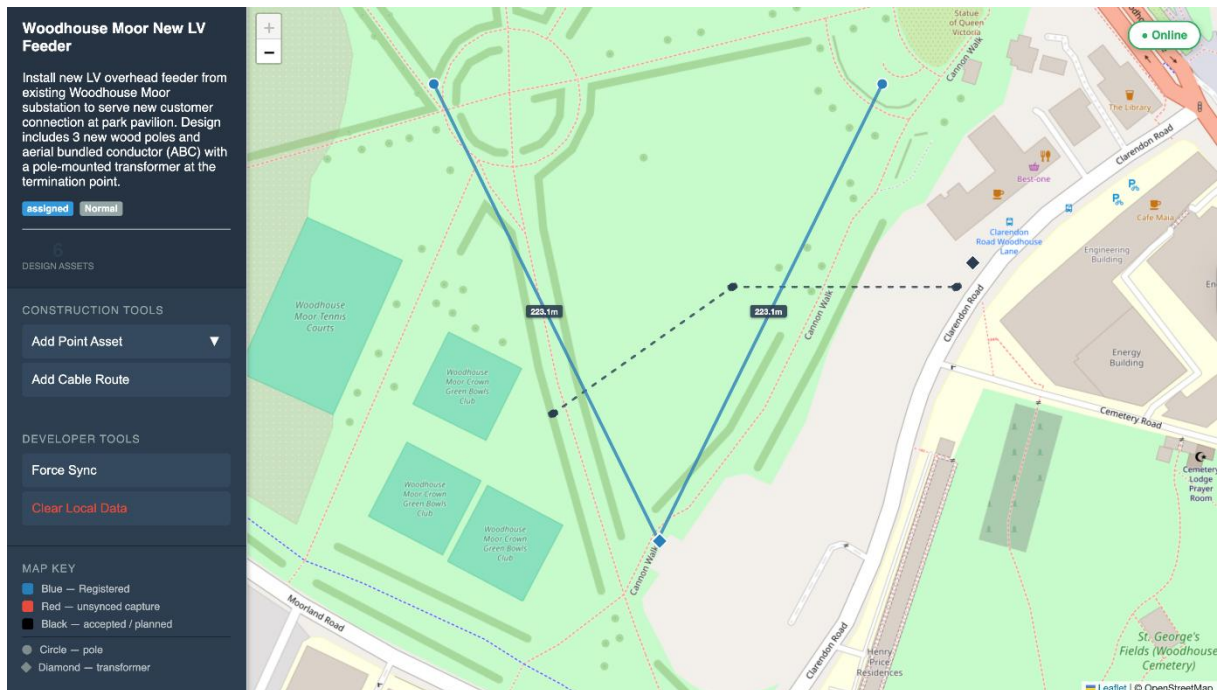


Figure 3.1: Screenshot of the running application: map interface loaded in browser.

The system is organised into two root directories within the repository: a frontend directory containing the Service Worker, frontend code and assets, and a backend directory containing the Flask server implementation and the rest of the backend code (SQLAlchemy models, API routes, and test suite). The full repository structure is provided in Appendix C. The *run.sh* shell script in the repository automatically starts the server and seeds the database upon execution. This allows for a clean, reproducible test environment.

3.2 Tools and Technologies

The full stack is summarised in Table 3.1. Selection rational was covered in Section 2.2; this table records the specific versions and roles used during implementation.

Component	Tool	Version	Role
Map Rendering	Leaflet.js	1.9.4 (CDN)	Interactive map, Layer Control
Client Storage	Dexie.js	4.0.11 (CDN)	IndexedDB wrapper
Offline Caching	Service Worker API	Native Browser	App-shell + tile cache
Backend Framework	Flask	>=2.0.0	REST API routing
CORS	Flask-CORS	>=3.0.0	CORS headers
ORM	Flask-SQLAlchemy	>=3.0.0	Schema enforcement, audit writes
Database	SQLite	Python stdlib	Central asset register

Table 3.1 – Technology stack with versions and roles.

3.3 Architectural Evolution and Prototyping

The system's development lifecycle purposefully front-loads the user interface and maps application development to create a working prototype. The early iterations of the software focused on integrating Leaflet.js for mapping, creating CNAIM-inspired data capture forms (Ofgem et al., 2017), and using Dexie.js to store data locally. To accelerate frontend delivery, the initial backend was created with minimal functionality: a basic Flask server that read from and wrote to a flat database file, *database.geojson*.

This provided an interim state to validate frontend mechanics before implementing a hardened solution, leaving these artefacts behind:

- The method used to synchronise the client's local GeoJSON collection with that stored on the server utilised a diffing algorithm to determine the changes required to synchronise the two collections (a brittle but simple approach discussed in Chapter 1).
- The system used a Boolean field (*pending_sync*) within the asset record to track offline changes. There was no transaction log to track changes made by the client while offline.
- The backend system used flat files to store GeoJSON data without a relational database schema or transaction safety.

- The frontend was monolithic in design and used a single JavaScript file containing 900 lines of code.

To satisfy the NFRs and Acceptance Criteria established in Chapter 1, a comprehensive architectural rewrite was conducted. The frontend was modularised into distinct services (API, DB, UI, Map), and the Service Worker was introduced to cache the application shell. Simultaneously, the backend was upgraded to an SQLite relational database managed by SQLAlchemy. Most critically, the state-diffing mechanism was entirely replaced with the robust Action Queue pattern, which formed the core of the final implementation.

Phase	Development Focus	Key Features Implemented
Phases 1& 2	Rapid Prototyping & Mapping	Basic UI shell, network connectivity polling, Leaflet.js integration, mock work orders, and map drawing tools (point/polyline) for asset redlining.
Phase 3	Data Quality and Modularisation	CNAIM-aligned forms, conditional input fields (e.g., GMT transformer attributes), offline photo capture, and frontend CSS/JS extraction.
Phase 4	Architectural Rewrite	IndexedDB Action Queue (<i>sync_queue</i>), Flask/SQLite relational backend, and atomic synchronisation with backend audit logging.
Final v2 (PoC)	System Hardening & Validation	Full PWA Service Worker integration for total offline isolation, live Admin Activity Dashboard, fault rollback handling, and comprehensive automated test suites.

Table 3.2 – Overview of development focus for each phase

3.4 Core System Implementation

Following the architectural rewrite of the PWA (Section 3.3), the system was implemented with three main components: the local persistence functionality (IndexedDB via Dexie.js), the Action Queue, and the Flask API backend (SQLite + SQLAlchemy).

3.4.1 Client-Side State and the Action Queue

The main component of the system that runs on the client is the class responsible for the local persistence of application data. Within the `db.js` file, three tables are defined:

local_work_orders, *local_assets*, and *sync_queue*.

In `app.js`, which is the main orchestrator script, two steps occur whenever an asset is edited while offline. First, the GeoJSON data is written directly into *local_assets*, allowing the Leaflet

map UI to reflect the change instantly. Assets fetched from the server are tagged *_source: "server"* before being saved to *local_assets*; locally-created assets are tagged *_source: "local"*, which *MapController* reads to apply the blue or red styles shown in Table 2.1. Next, the system serialises the user action that occurred, representing it as an action object in the *sync_queue*.

The *sync_queue* serves as a log of actions that occur on the client while the engineer is offline. Rather than employing state-diffing or isolated Boolean flags, this Action Queue acts as the transaction log for each action to be performed upon sync. Each entry in the queue contains two components: the type of action to be performed and the payload associated with that action. For instance, each entry for creating a new asset looks like:

```
1. const assetId = AppState.editingAssetId || `temp-${Date.now()}`;
2. const isUpdate = !!AppState.editingAssetId;
3. await DB.queueAction({
4.   action: isUpdate ? "UPDATE" : "CREATE",
5.   asset_id: assetId,
6.   work_order_id: woId,
7.   asset_type: assetType,
8.   geometry: AppState.pendingAssetGeometry,
9.   properties,
10. });
```

The *sync_queue* uses an auto-incrementing primary key (*++id*) to identify the items appended to the Action Queue; therefore, the order in which the sync operations are performed is chronological. Additionally, using a unique *temp-\${Date.now()}* string as the *asset_id* for each sync request ensures that even if the sync request is made multiple times due to bad network conditions, the backend will not create duplicate assets.

To automatically calculate cable length, the Haversine function is used. The *calculateLineLength()* function calls *haversineDistance()* to compute the length of the *LineString* geometry representing each cable. The length of each cable is shown as a label at its centre (the formula used to calculate the cable lengths is given in Appendix C, Equation C.4.1).

3.4.2 The Synchronisation Engine

To monitor network availability, the system actively polls the backend */api/health* endpoint via a GET request, as defined in *api.js* and invoked in *app.js*. Each poll request is wrapped in an *AbortController* with a 2,000 ms timeout to prevent a slow server from blocking the UI thread. This endpoint also returns a *boot_id* (UTC timestamp generated at Flask process initialisation). If the returned value differs from the one stored in *localStorage*, the client automatically purges *local_work_orders* and *local_assets* but deliberately preserves the *sync_queue*, as pending actions represent field intent that cannot be re-fetched. Upon detecting a transition from offline to online, the application's status badge is updated (see Appendix C Figure C.1.13 Sequence

Diagram), at which point the engineer can manually trigger the *syncOfflineChanges()* function to transmit the changes to the server.

The synchronisation engine retrieves the entire *sync_queue* array from the application's database and sends it to the *backend server's POST /api/sync endpoint*. This minimises the number of requests made over high-latency networks. Upon receiving an *HTTP 200* status from the server (which is returned for all successfully processed batches of actions), the response's results array is examined. If the top-level status code from the server was 200, the *DB.clearSyncQueue()* routine is triggered. While actions can fail individually (as indicated by an entry in the results array with the value *{ "status": "error" }*), those errors are logged to the server console and do not prevent the database from clearing the *sync_queue*.

3.4.3 Backend API and Relational Integrity

The final stage of the offline-first architecture is the Python backend. The backend enforces schema integrity and provides an immutable audit trail for all field interactions.

Within *models.py*, the database schema is defined. Three tables are defined: *work_orders*, *assets*, and *audit_log*. The *assets* table serves as the master register for all as-built infrastructure; the *work_order* table manages the planned design assets that are pushed out to Field Technicians.

When the *POST /api/sync* endpoint receives a batch of operations from the client's Action Queue, it iterates through the actions sequentially, using LWW as the conflict resolution model

For each valid action (*CREATE*, *UPDATE*, *DELETE*, *ACCEPT*), the server executes the corresponding ORM command. A special case is *DELETE*. This action does not physically remove a row from the database; instead, the application soft-deletes the record by updating its state, mimicking an asset decommissioning rather than a hard delete. The *ACCEPT* action additionally mutates the parent work order's *design_assets* JSON array to mark the accepted design as *"accepted"*, ensuring the job pack and asset register remain consistent.

In addition to mutating the *assets* and *work_order* records, every single operation processed by the sync endpoint writes a corresponding record to the *audit_log* table. This log captures the action type, the targeted *asset_id*, the timestamp, and the full JSON snapshot of the mutation payload:

The use of SQLAlchemy's *db.session.commit()* ensures that the asset mutation and its corresponding audit log are written to the database together at the conclusion of the batch. If an exception occurs while processing a specific action, such as a malformed payload, the server catches the error, flags that *asset_id* as failed in its response array and continues processing the remaining valid actions in the queue.

3.5 System Validation and Testing

To ensure the system fulfils the acceptance criteria (Section 1.6), testing was performed at three levels: automated backend API testing, frontend integration testing, and manual simulation of the system's offline workflow.

3.5.1 Backend API Validation

The backend API and database were tested using a suite of unit tests implemented in *test_backend.py*. The results of these tests (Figure C.3.2 in Appendix C) indicate that the API and database are functioning as expected. The Flask routing and SQLAlchemy ORM successfully enforce the relational schema of the asset register. Furthermore, structural database schema constraints (e.g., non-nullable column violations) are enforced by SQLAlchemy at the model definition level in *models.py*, rather than being explicitly exercised by the test suite. This is most certainly an area of improvement, and the robustness of database transactions and the additional data checks that could be implemented will be discussed later.

3.5.2 Frontend Integration Testing

For the frontend section of the PWA to interact with APIs exposed by the backend server, it utilises browser-provided APIs. Consequently, a suite of tests written for the browser (*system-checks.js*) was implemented. These tests were performed in the browser console (Figure C.3.1 in Appendix C) to verify the integration of the PWA with the IndexedDB cache, confirm that the asset data capture forms generate the correct GeoJSON strings, and confirm that local data mutations are appended to the Action Queue.

3.5.3 Manual Offline Workflow Simulation

To test and demonstrate the system's offline functionality, manual testing was performed. During this test, the server process was terminated while the tester was actively performing field work within the system. This test validated the Action Queue functionality, Service Worker app shell caching, and offline drawing functionality. Figures C.1.2 and C.1.3 in Appendix C illustrate the changes to the system's UI during a network error. The "Offline Functionality Demonstration" video includes comparisons against the requirements (Section 1.5.1 for functional requirements and Section 1.5.2 for non-functional requirements).

Chapter 4

Results, Evaluation and Discussion

4.1 Evaluation Against Acceptance Criteria

The acceptance criteria defined in Section 1.6 were used to evaluate the PWA's effectiveness. The validation relied on qualitative test outputs from simulated use of the system and PASS/FAIL tests, as discussed in Section 3.5. Unfortunately, due to time constraints and complications, detailed multi-user testing and real-world field scenarios were not conducted.

4.1.1 Scenario A: Offline Asset Capture and Queuing

Verdict: PASS

The system decoupled data entry from network availability. Tests show that when an asset form is submitted offline, it is serialised to GeoJSON and saved in IndexedDB. Additionally, a CREATE operation is added to the Action Queue with a unique key. The UI reflects the change without any network or connection errors.

4.1.2 Scenario B: Deterministic Synchronisation upon Reconnection

Verdict: PASS

This scenario uses the tests to demonstrate that the server processes actions deterministically when the client reconnects to the network. The server acknowledges the actions sent over the network (*HTTP 200* responses). Upon receiving these acknowledgements, the client purges the Action Queue entries from IndexedDB, returning the UI to the zero-pending-action state for all assets.

4.1.3 Scenario C: Error Handling and Queue Isolation

Verdict: Partial Pass

This scenario shows that errors in the database from individual assets do not impact the payload of data sent to the server. Tests were performed on the backend that sent invalid actions to the database. The API caught these invalid actions and did not crash the server or database, allowing for a resubmission. However, due to development time constraints, deep transaction isolation using nested SQLAlchemy savepoints was descoped from the current iteration. The architectural fix, implementing explicit session rollbacks for individual failed actions, is already understood and would be prioritised in future development.

4.2 Objectives and Requirements Review

To measure the overall success of the proof-of-concept, the delivered system was evaluated against the formal objectives, NFRs, and MoSCoW criteria given in Chapter 1.

4.2.1 Achievement of Project Outcomes

The project successfully delivered on its core aims, providing the beginnings of a viable alternative to heavy enterprise vendor lock-in for offline field capture. The specific outcomes for each objective are detailed below:

Objective	Description	Outcome & Evidence
1 & 2	Analyse connectivity constraints and define persona-driven requirements.	Achieved: Literature review (1.3) and persona synthesis (1.4) directly informed the decoupled architecture and BDD scenarios.
3	Architect and implement offline-first caching.	Achieved: The Service Worker successfully intercepts network requests, serving the application shell and map tiles without server dependency.
4	Implement an Action Queue and redlining workflow.	Achieved: A durable IndexedDB queue serialises all operations. Spatial geometries can be modified in-place to create as-built records.
5	Enforce data quality at the point of entry.	Achieved: CNAIM-aligned forms prevent queue commitment unless mandatory fields and valid photo captures are present.
6	Evaluate system resilience and usability.	Achieved: <i>system-checks</i> and <i>pytest</i> suites, alongside manual network simulation, validated data durability during offline usage (4.1).

Table 4.1: Comparison of the initial project aims with the final project achievements.

4.2.2 Non-Functional Requirements (NFR) Validation (Demo Video Tests)

The engineering constraints of the offline-first architecture were tested during runtime:

NFR1 (Data Persistence): **Met**. Even after terminating the browser process, the work order data persisted in IndexedDB

NFR2 (UI Responsiveness): **Met.** The application gracefully transitioned between online and offline modes without affecting the main application thread or the map engine.

NFR3 (Storage Efficiency): **Met.** Using GeoJSON files to store network topology data resulted in small file sizes that respected browsers' storage limits.

NFR4 (Auditability): **Met.** Every action written to the SQLite database had a timestamp and an idempotency key to allow auditing of changes.

4.2.3 Functional Requirements Delivery Assessment

The scope of the project was managed throughout its development. All the features required to be “Must Have” for the application to be considered complete were included. Additionally, all high-priority “Should Have” and “Could Have” features were included. More details are in the associated “Offline Functionality Demonstration”.

4.3 Discussion

4.3.1 Architectural Trade-offs

One of the main design choices was the use of a Last-Write-Wins strategy to resolve conflicts that may arise when multiple devices write to the database. As discussed in Section 1.3.3, Last-Write-Wins strategies are inherently risky in safety-critical contexts. However, there is an argument to be made that this may not be an issue for this application’s use case, as it is likely that one field worker or engineer will be making edits at a time. The shortcomings of LWW would be more of a problem when multiple assets exist across multiple work orders, which was out of scope for this proof of concept. If the system is developed for multi-user deployments, a strategy such as Conflict-free Replicated Data Types (CRDTs) should be implemented to ensure that edits from multiple users are resolved without corrupting data.

A second trade-off was using SQLite instead of a database like PostGIS, which provides spatial database functionality often required for DNO operations. Using SQLite allowed rapid setup without heavy configuration. However, as with the conflict-resolution approach, using SQLite for the proof of concept would likely need to change for live DNO deployments, which require spatial database functionality.

4.3.2 Proof-of-Concept Scope and Limitations

When reviewing the application's features and capabilities, it is important to contextualise the achievements within the scope of the proof of concept. The system has achieved the goal of preventing data loss when connectivity is lost and regained by effectively queuing actions that are applied to the central asset register upon re-establishment of connectivity. It does

not, however, feature the additional functionality expected of an enterprise GIS system, as these represent a significant step up in complexity and development time. The system lacks a comprehensive asset taxonomy and the ability to create/add custom assets. While an improvement over the initial iterations, the backend is still somewhat brittle and would require further development to achieve production-ready robustness.

4.4 Conclusion

The project successfully created an offline-first PWA that prevents data loss in low-connectivity environments. By replacing legacy, occasionally connected GIS software with the browser-based, offline-first design, the system ensures that changes to network assets are not lost due to connectivity issues. The PWA successfully caches the network topology for the area of interest, allows offline redlining and blackline design confirmation of assets, and synchronises changes to a relational database upon reconnection, thereby isolating transaction faults and maintaining full operability while still offline.

While the current proof-of-concept is intentionally single-user and supports a limited asset taxonomy, it demonstrates that standard web technologies (Service Workers, IndexedDB, and GeoJSON) can be combined to address the fragile last-mile data capture problem faced by DNOs. It provides a lightweight, vendor-independent foundation that prioritises Field Technician autonomy without compromising back-office data integrity.

4.5 Future Work

As an immediate next step before pursuing functional enhancements, the prototype should be migrated from a local development environment to a managed cloud platform such as Microsoft Azure. Deploying the Python backend to a scalable environment and transitioning the SQLite data store to a production-grade spatial database (e.g., Azure Database for PostgreSQL with PostGIS) are prerequisites for enterprise readiness. This infrastructure shift establishes high availability, secure authentication gateways (e.g., via Microsoft Entra ID), and concurrent write handling, all necessary for the following stages of development.

4.5.1 Multi-User Concurrent Conflict Resolution

The most critical requirement for scaling this architecture is transitioning from a Last-Write-Wins (LWW) synchronisation model to one that can handle concurrent, multi-user edits. Future iterations should explore implementing Conflict-free Replicated Data Types (CRDTs). By embedding conflict-resolution semantics directly into the JSON data structures, CRDTs would allow multiple field technicians to edit the same network geometries or asset attributes

simultaneously, offline, and guarantee a consistent global state upon synchronisation without requiring centralised coordination.

4.5.2 Enterprise Integration and ETL Pipelines

Currently, the system's backend API uses a standalone SQLite database. To provide value to a DNO, the system must be able to connect to an enterprise database containing a DNO's assets register. An ETL process should be developed to transform GeoJSON data from the Action Queue system into the enterprise system's more complex, proprietary database schemas. Such a process would require mapping the forms' fields and data to the relevant tables in an enterprise asset management database (e.g., IBM Maximo), and transforming coordinates from the web standard system (such as WGS84) to the local grid system (such as the British National Grid).

4.5.3 Map Tile Caching at Scale

Currently, the Service Worker caches vector and raster data for the maps. Future development in this area can focus on optimising the system to store offline maps in greater detail, with more GIS functionality within the standard browser storage limit. For example, limits can be applied to the bounding boxes of the maps downloaded and cached by users, restricting them to caching only specific zoom levels of their work order areas.

4.5.4 Native Application Migration

Currently, the application is implemented as a PWA that runs in a browser. However, browser limitations and Service Worker storage constraints may prevent the same features from functioning in a native mobile application. Therefore, migrating the Action Queue system to a native mobile application framework (such as React Native or Swift) would allow the application to bypass browser limitations, utilise embedded databases on mobile devices, and maintain the existing vendor-agnostic system.

4.5.5 Enhancing Drawing Capabilities and Asset Taxonomy

To make sure this system can deliver the standard features required by a DNO managing grid assets, the drawing tools would need to be enhanced. An immediate step for this platform would be to implement Leaflet.Draw, which enables touch-input drawing for linear assets, replacing the current point-to-point click implementation. For further development, a comprehensive suite of drawing and mark-up tools would be required. Furthermore, the existing asset taxonomy is limited to cables, transformers and poles. This is not representative of a complete asset catalogue required to manage a grid. DNOs would also benefit if the PWA supported the addition of custom assets.

List of References

1. 1Spatial 2021. The Service-based Architecture of ArcGIS Utility Network - 2021 | 1Spatial. Available at: <https://1spatial.com/news/the-service-based-architecture-of-arcgis-utility-network-2021/> (Accessed: 2025-11-13).
2. Adobe Experience Cloud Team 2023. Progressive Web Apps — 12 Examples of PWAs Done right. Adobe for Business. [Online]. Available from: <https://business.adobe.com/blog/basics/progressive-web-app-examples> (Accessed: 2025-10-12)
3. Agafonkin, V. 2025. Leaflet — an open-source JavaScript library for interactive maps. Leaflet. [Online]. Available at: <https://leafletjs.com/> (Accessed: 2025-10-12).
4. Android Developers 2026. Build an offline-first app. Android Developers. [Online]. Available at: <https://developer.android.com/topic/architecture/data-layer/offline-first> (Accessed: 2025-10-12).
5. Archibald, J. 2014. The offline cookbook. JakeArchibald.com. [Online]. Available at: <https://jakearchibald.com/2014/offline-cookbook/> (Accessed: 2026-02-13)
6. Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S. and Schaub, T. 2016. The GeoJSON format [Online]. Available at: <https://www.rfc-editor.org/rfc/rfc7946> (Accessed: 2026-03-04).
7. Chapungo, N.J. and Postolache, O. (2025) 'Experimental evaluation of LoRaWAN connectivity reliability in remote rural areas of Mozambique', Sensors, 25(19), 6027. Available at: <https://doi.org/10.3390/s25196027> (Accessed: 2026-02-12)
8. Clifford, J. and Curry, D. (2021) 'Work offline with Field Maps', ArcGIS Blog. Available at: <https://www.esri.com/arcgis-blog/products/field-maps/field-mobility/work-offline-with-field-maps> (Accessed: 2025-12-02).
9. Damilola, D.-A. (2013) 'Geospatial modeling of electricity distribution network', Geospatial World. Available at: <https://geospatialworld.net/article/geospatial-modeling-of-electricity-distribution-network> (Accessed: 2025-12-04).
10. Edwards, W.K., Mynatt, E.D., Petersen, K., Spreitzer, M.J., Terry, D.B., Theimer, M.M., and Xerox Palo Alto Research Center 1997. Designing and implementing asynchronous collaborative applications with Bayou [Online]. Association for Computing Machinery. Available at: <https://faculty.cc.gatech.edu/~keith/pubs/bayou.pdf> (Accessed: 2026-03-07).
11. Distributed System Authority 2026. Idempotency and Exactly-Once semantics in distributed systems. Distributed System Authority. [Online]. Available at: <https://distributedsystemauthority.com/idempotency-and-exactly-once-semantics> (Accessed: 2026-04-02).

12. Esri 2026. Field Data Collection App for mobile workers | ArcGIS Field Maps. Esri. [Online]. Available at: <https://www.esri.com/en-us/arcgis/products/arcgis-field-maps/overview> (Accessed: 2026-04-02).
13. eSmart Systems (2024) '3 simple steps to uplift the quality of your grid asset data', eSmart Systems. Available at: <https://www.esmartsystems.com/news-and-resources/3-simple-steps-to-uplift-the-quality-of-your-grid-asset-data> (Accessed: 2026-04-01).
14. Firtman, M., Andrews, R., Jara, A., LePage, P., Richard, S. and Medley, J. 2021. Welcome to learn progressive web apps! | web.dev. web.dev. [Online]. Available at: <https://web.dev/learn/pwa/welcome> (Accessed: 2025-11-13).
15. Fertig, T. and Braun, P. 2018. Towards offline support for RESTful systems. WWW '18: Companion Proceedings of the Web Conference 2018., pp.1875–1880. Available at: <https://doi.org/10.1145/3184558.3191655> (Accessed: 2026-03-12).
16. Fulcrum (2026) 'Electric utilities | field data collection software', Fulcrum. Available at: <https://www.fulcrumapp.com/industries/electric-utilities> (Accessed: 2026-04-10).
17. Geoapify 2019. Leaflet vs OpenLayers: Pros and Cons of both libraries | Geoapify. Geoapify. [Online]. Available at: <https://www.geoapify.com/leaflet-vs-openlayers/> (Accessed: 2025-10-12).
18. Globema (2019) 'Smallworld Electric Office – electric utility asset management solution', Globema. Available at: <https://www.globema.com/portfolio/smallworld-electric-office> (Accessed: 2025-11-08).
19. Groden, A. (2025) 'Offline + sync architecture for field operations: guide & tools', Alpha Software blog. Available at: <https://www.alphasoftware.com/blog/offline-sync-architecture-tutorial-examples-tools-for-field-operations> (Accessed: 2026-01-20).
20. Guda, V.R. 2025. Implementation of Offline-First Architectures for Android Internet-Based Chat Systems. International Journal of Multidisciplinary Research and Growth Evaluation. 6(3), pp.1200–1203. Available at: https://www.allmultidisciplinaryjournal.com/uploads/archives/20250607132526_MGE-2025-3-252.1.pdf (Accessed: 2025-10-12)
21. Hartung, C., Lerer, A., Anokwa, Y., Tseng, C., Brunette, W. and Borriello, G. 2010. Open Data Kit: Tools to build information services for developing regions. Proceedings of the 4th ACM/IEEE International Conference on Information and Communication Technologies and Development., pp.1–12. Available at: <https://dl.acm.org/doi/10.1145/2369220.2369236> (Accessed: 2026-04-02).
22. Frank, C., Larreal, A. and Henderson, D. 2025. (2025) 'Improving asset risk mitigation by making data a top priority', Utility Analytics Institute. Available at:

- <https://utilityanalytics.com/improving-asset-risk-mitigation-by-making-data-a-top-priority> (Accessed: 2026-03-15).
23. Halounová, L. and Department of Mapping and Cartography, Faculty of Civil Engineering, Czech Technical University in Prague 2007. Freeware for GIS and remote sensing. Available at: <https://ojs.cvut.cz/ojs/index.php/gi/article/download/gi.2.7/2582> (Accessed: 2026-02-14).
24. Kaczmarczyk, A., Zając, P. and Zabierowski, W. 2022. Performance comparison of native and hybrid Android mobile applications based on sensor Data-Driven applications based on Bluetooth low energy (BLE) and Wi-Fi communication architecture. *Energies*. 15(13), p.4574. Available at: <https://www.mdpi.com/1996-1073/15/13/4574> (Accessed: 2026-02-15).
25. Kavlakoglu, E. and IBM (2025) 'Agile vs waterfall', IBM Think blog. Available at: <https://www.ibm.com/think/topics/agile-vs-waterfall> (Accessed: 2026-02-20).
26. Kenny, A., Gordon, N., Downey, J., Eddins, O., Buchholz, K., Menyon, A. and Mansah, W. 2020. Design and implementation of a mobile health electronic data capture platform that functions in fully-disconnected settings: a pilot study in rural Liberia. *BMC Medical Informatics and Decision Making*. 20(1), p.39. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7036217/> (Accessed: 2026-02-14).
27. Kleppmann, M., Wiggins, A., Van Hardenberg, P. and McGranaghan, M. 2019. Local-First Software: You Own Your Data, in spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '19)*, October 23–24, 2019, Athens, Greece. ACM, New York, NY, USA, 25 pages. Available at: <https://doi.org/10.1145/3359591.3359737> (Accessed: 2026-04-03).
28. Kleppmann, M., Beresford, A.R., and University of Cambridge Computer Laboratory 2017. A Conflict-Free replicated JSON datatype. *arXiv*. Available at: <https://arxiv.org/pdf/1608.03960> (Accessed: 2026-04-03).
29. Krebs, B. and Martinez, J.C. 2025. Developing RESTful APIs with Python and Flask. Auth0 - Blog. [Online]. Available at: <https://auth0.com/blog/developing-restful-apis-with-python-and-flask/> (Accessed: 2026-02-15).
30. McKelvey, J. (2025) 'How to build robust offline-first apps: a technical guide to conflict resolution with CRDTs and Ditto', Ditto. Available at: <https://www.ditto.com/blog/how-to-build-robust-offline-first-apps-a-technical-guide-to-conflict-resolution-with-crdts-and-ditto> (Accessed: 2026-03-24).
31. MDN 2025. Caching - Progressive Web Apps | MDN. MDN. [Online]. Available at: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Caching (Accessed: 2026-02-15).

32. Muresan, D. (2024) 'Take nonversioned data offline', ArcGIS Blog. Available at: <https://www.esri.com/arcgis-blog/products/arcgis-pro/data-management/take-nonversioned-data-offline> (Accessed: 2025-12-03)
33. Nguyen, M.Q., Tran, D.T., An Giang University, and Quy Nhon University 2025. Managing Conflict Resolution and Version Control in Bi Directional Data Environments. International Journal of Computational Methods and Applied Sciences. 15(10). Available at: <https://scisearch.net/index.php/IJCMAS/article/view/Quant2025> (Accessed:2026-04-03).
34. Ofgem, Bell, D., Kirkmorris, E., and System Flexibility 2025. Consultation Enhancing asset visibility: Distribution Network Operator Options [Online]. Available from: <https://www.ofgem.gov.uk/sites/default/files/2025-12/Enhancing%20asset%20visibility%20-%20Distribution%20Network%20Operator%20Options%20consultation.pdf> (Accessed: 2026-04-10)
35. Ofgem, Booth, J., Wells, B., Seeds, Black, M., Howarth, Nickolson, M., Boyd, Sherwood, P., Hurley, J., Sharma, R., Johnston, L., Smart, J., Butler, I., Friel, R., Wakelen, R., Mann, P. and Tighe, D. 2017. DNO COMMON NETWORK ASSET INDICES METHODOLOGY [Online]. Ofgem. Available at: https://www.ofgem.gov.uk/sites/default/files/docs/2017/05/dno_common_network_asset_indices_methodology_v1.1.pdf (Accessed: 2025-11-11).
36. ProductPlan (2025) 'What is MoSCoW prioritization? Overview of the MoSCoW method', ProductPlan. Available at: <https://www.productplan.com/glossary/moscow-prioritization> (Accessed: 2026-03-05).
37. Raju, S. and Oracle (2026) Oracle Fusion Field Service: Using core application. Oracle. Available at: <https://docs.oracle.com/en/cloud/saas/field-service/faaca/using-core-application.pdf> (Accessed: 2026-04-13).
38. Roberts, M. (2025) '4 best practices to transform digital as-built capture for electric utilities', IQGeo blog. Available at: <https://www.iqgeo.com/blog/4-best-practices-to-transform-digital-as-built-capture-for-electric-utilities> (Accessed: 2026-03-17).
39. Saito, Y. and Shapiro, M. 2005. Optimistic replication. ACM Computing Surveys. 37(1), pp.42–81. <https://dl.acm.org/doi/10.1145/1057977.1057980> (Accessed: 2026-02-19).
40. Sanderson, L. (2025) 'Join fields in Smallworld', FME Support Center. Available at: <https://support.safe.com/hc/en-us/articles/25407625490829-Join-Fields-in-Smallworld> (Accessed: 2026-04-18).
41. Ying, F. 2013. Management of spatial data for visualization on mobile devices [Online]. Available from: <https://mural.maynoothuniversity.ie/id/eprint/4386/> (Accessed: 2026-03-13).

42. Shaner, J., Curry, D.M. and Pierce, B. (n.d.) Introduction to ArcGIS Field Maps. Esri. Available at: <https://www.esri.com/content/dam/esrisites/en-us/events/conferences/2021/developer-summit/12348-introduction-to-field-maps.pdf> (Accessed: 2026-03-30)
43. Sirigiri, K., Narra, A. and Sirumalla, A. (2023) 'Offline-first PWA: case study on efficient data handling and synchronization', International Journal of Scientific Research in Computer Science Engineering and Information Technology, 9(3), pp. 747–754. Available at: <https://doi.org/10.32628/CSEIT25112782> (Accessed: 2025-12-13)
44. Sinha, A. and Oracle (2026) Oracle Fusion Field Service Mobile Plugin Framework. Oracle. Available at: <https://docs.oracle.com/en/cloud/saas/field-service/fapcf/mobile-plugin-framework.pdf> (Accessed: 2026-03-10).
45. Smulders, C. 2022. Maximizing your Offline Success with ArcGIS Field Maps. Esri Canada. [Online]. Available at: <https://resources.esri.ca/getting-technical/maximizing-your-offline-success-with-arcgis-field-maps> (Accessed: 2026-03-15).
46. Stichbury, J. (2022) 'CRDTs solve distributed data consistency challenges', Ably Realtime blog. Available at: <https://ably.com/blog/crdts-distributed-data-consistency-challenges> (Accessed: 2026-03-27).
47. Tossani, F. (2018) 'Conflict resolution: using last-write-wins vs. CRDTs', DZone. Available at: <https://dzone.com/articles/conflict-resolution-using-last-write-wins-vs-crdts> (Accessed: 2026-01-26).
48. Trogrlic, I. 2023. 8 biggest challenges of Android app development. DECODE. [Online]. Available at: <https://decode.agency/article/android-app-development-challenges/> (Accessed: 2025-12-13).
49. Wan, X. (2016) 'A geographic information system-based modelling, analysing and visualising of low voltage networks: the potential of demand time-shifting in the power quality improvement', MSc thesis, Uppsala University. Available at: <https://www.diva-portal.org/smash/get/diva2:1709127/FULLTEXT02.pdf> (Accessed: 2026-04-07).
50. Gao, S. and Mai, G. (2018) Mobile GIS and location-based services. University of Wisconsin–Madison. Available at: https://geography.wisc.edu/wp-content/uploads/sites/28/2022/05/2018_MobileGISLBS.pdf (Accessed: 2026-03-27).

Appendix A

Self-appraisal

A.1 Critical self-evaluation

This project was motivated by direct industry experience working with a DNO facing a similar problem. Having seen how enterprise GIS tools fail in low-connectivity environments, I identified this as a computer science problem centred on distributed data synchronisation. The problem space was broad, so I scoped it to two specific challenges: offline caching of work orders and deterministic syncing of data changes upon reconnection.

In large part, I met my aims:

- **Architecture:** I designed and implemented an offline-first PWA using Service Workers, IndexedDB, and a transparent Action Queue.
- **Data Integrity:** The synchronisation engine enforces idempotent operations and fault isolation, ensuring no data loss.
- **Validation:** The system was tested via automated backend testing (`test_backend.py`), frontend integration checks (`system-checks.js`), and manual network partition simulation.

However, I began coding too early. Light, informal research led directly to tinkering, without conducting proper research into the offline-first paradigm or available web technologies. This forced a significant architectural rewrite mid-project, specifically integrating Service Workers and transitioning to a relational SQL structure. Despite this, anticipating that the development would not be straightforward, I had built sufficient buffer time into my schedule from the outset, allowing me to complete the rewrite.

A.2 Personal reflection and lessons learned

Reflecting on the technology stack, I decided to build the system using languages I was already proficient in, Python and JavaScript. While a native application using Kotlin or Java would have been more impressive, robust and feature-rich, especially for mobile hardware, learning a new mobile framework from scratch would have consumed the majority of the project timeline. Sticking to a PWA allowed me to focus purely on solving the core synchronisation logic rather than fighting unfamiliar syntax.

The most significant lesson I take away from this project is the danger of the "code-first" approach. My rush to prototype meant I was designing the architecture in real time, leading to fragile initial iterations. If I were to do this project again, I would change my methodology. I

would conduct a strict, comprehensive design phase before writing any production code. This would involve conducting a much deeper literature review of offline-first patterns and conflict-resolution strategies at the very beginning, so that the architectural blueprint would be robust enough to handle the problem domain before the implementation phase began. Learning to slow down and value the planning and research phases as highly as the development phase is the most valuable professional growth I have gained from this module.

While conducting research, it became clear that there are already incredibly robust solutions to this problem. Taking a novel approach, while enjoyable to research and push the boundaries of my computer science capabilities, I doubt my solution could be developed further to replace existing systems. However, completing this project also taught me something else: the power of open-source software. I have, to a certain extent, demonstrated that alternatives to vendor-specific proprietary software are possible by developing a vendor-independent framework to address some of the core issues faced by in-field asset data capture. That being said, it is not lost on me just how much time, effort and resources are required to build a system such as Esri's ArcGIS. The safety-critical domain in which this type of software operates adds another level of responsibility to development, and I recognise that much more goes on behind the scenes to ensure the operation of these systems works seamlessly, where possible, beyond simple application development.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

The main legal issue in this project was the use of third-party software libraries and frameworks within the implementation. As the system was built using technologies such as Flask, SQLAlchemy, browser APIs, and Dexie.js, it was necessary to use these tools in accordance with their licenses and standard terms of use.

There was little additional legal risk in the project itself because the work was a proof-of-concept focused on architecture, local storage, synchronisation logic, and validation, rather than the handling of live customer records or formal data collection from participants.

A.3.2 Social issues

The main social issue was usability in environments with poor connectivity. The project was motivated by the fact that field systems can become unreliable under remote working conditions, so improving offline capture and recovery from connection loss has a direct social benefit for the people who are expected to use such software in practice, as well as improving grid reliability to customers.

A.3.3 Ethical issues

In this project, validation was carried out through automated backend tests, browser-based integration checks, and manual workflow simulation, so the usual issues around participant consent and withdrawal were not a factor.

The clearest ethical issue that remains is the responsibility to the Field Technicians working in dangerous environments. Tools need to be reliable and not cause additional distraction or stress in these environments. So, putting the end user first during development is critical.

A.3.4 Professional issues

Throughout the project, I adhered to the principles of professional competence and due care outlined in the BCS Code of Conduct. This was demonstrated by prioritising standard good practice in software engineering over quick fixes. Specifically, I maintained a strict separation of concerns by decoupling the PWA presentation layer from the backend APIs and local persistence logic.

Furthermore, the BCS Code of Conduct requires professionals to ensure the systems they develop are reliable, secure, and thoroughly tested. To meet this standard, validation mechanisms were embedded directly into the project structure. The use of automated testing scripts, alongside GitHub for version control, ensured that the system's core offline synchronisation logic was professionally tested and verified before any conclusions were drawn.

Appendix B

External Materials

B.1 Frontend Libraries

Material	Version	Source / URL	Usage
Leaflet.js	1.9.4	unpkg.com/leaflet@1.9.4	Interactive map rendering; tile layer; circle markers, polylines, popups, divIcons, and layer groups for asset display
Dexie.js	4.0.11	unpkg.com/dexie@4.0.11	IndexedDB wrapper providing the three offline object stores: local_work_orders, local_assets, and sync_queue

B.2 Backend Libraries

Material	Version	Source / URL	Usage
Flask	>=2.0.0	https://github.com/pallets/flask.git	Core web framework serving the REST API (8 routes) and the admin dashboard HTML
Flask-CORS	>=3.0.0	https://github.com/corydolphin/flask-cors.git	Cross-Origin Resource Sharing headers, allowing the browser frontend to call /api/* endpoints
Flask-SQLAlchemy	>=3.0.0	https://github.com/pallets-eco/flask-sqlalchemy	ORM integration; provides db instance and model declarations for WorkOrder, Asset, and AuditLog
SQLAlchemy	Installed via Flask-SQLAlchemy	PyPI	Marks mutated JSON columns as dirty in the ORM session during the ACCEPT sync action

SQLite	Built into Python stdlib	Python standard library	Relational database storing the three tables (work_orders, assets, audit_log) in instance/app.db
--------	-----------------------------	----------------------------	---

B.3 External Data & Tile Service

Material	Version	Source / URL	Usage
OpenStreetMap tile layer	Live/cached	https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png	Base map tiles rendered by Leaflet; cached offline by the Service Worker using a stale-while-revalidate strategy
GeoJSON (RFC 7946)	IETF RFC 7946, 2016	Butler, H. et al., datatracker.ietf.org/doc/html/rfc7946	The data format used for all geometry (assets, design assets, seed data in database.geojson) throughout the system
CNAIM	2017	Energy Networks Association / Ofgem	Used to define field names, condition grades, and inspection categories used in the pole, transformer, and cable data capture forms

B.4 Browser / Platform APIs

Material	Version	Source / URL	Usage
Service Worker API	Web standard	Native browser API (W3C)	sw.js registers a Service Worker that pre-caches static assets and map tiles, enabling full offline loading.
Cache API	Web standard	Native browser API (W3C)	Used within sw.js to store and retrieve cached responses (static shell + tile cache named offline-gis-v1)
IndexedDB	Web standard	Native browser API (W3C)	The underlying browser storage engine that Dexie.js abstracts; stores work orders, assets, and the sync queue

Fetch API	Web standard	Native browser API (W3C)	Used in api.js as the HTTP communication layer for all REST calls to the Flask backend
FileReader API	Web standard	Native browser API (W3C)	Used in the photo capture field of asset forms to convert images to Base64 for offline storage in IndexedDB
Web App Manifest (manifest.json)	W3C standard	Native browser/PWA standard	Enables the application to be installed as a standalone PWA on a device

B.5 Development Tools

Software	Version	Source / URL	Usage
pytest	Standard Python test runner	PyPI	Runs the backend tests
Live Server	-	Visual Studio Code Marketplace	Recommended in the README for serving the frontend directory over HTTP during development
Draw.io	-	https://app.diagrams.net/	Diagramming
Google Chrome	147.0.7727.102 (Official Build) (64-bit)	Google Chrome https://www.google.com/chrome/	Browser used to run, test and demo the PWA
iMovie	-	Apple Inc https://www.apple.com/imovie	Video editing

B.6 Generative AI Acknowledgement

I acknowledge the use of GitHub Copilot (Anthropic, <https://github.com/features/copilot>) to assist with code commenting, refactoring, and README maintenance for this project.

Appendix C

Supplementary Figures

C.1 Supplementary Figures UI

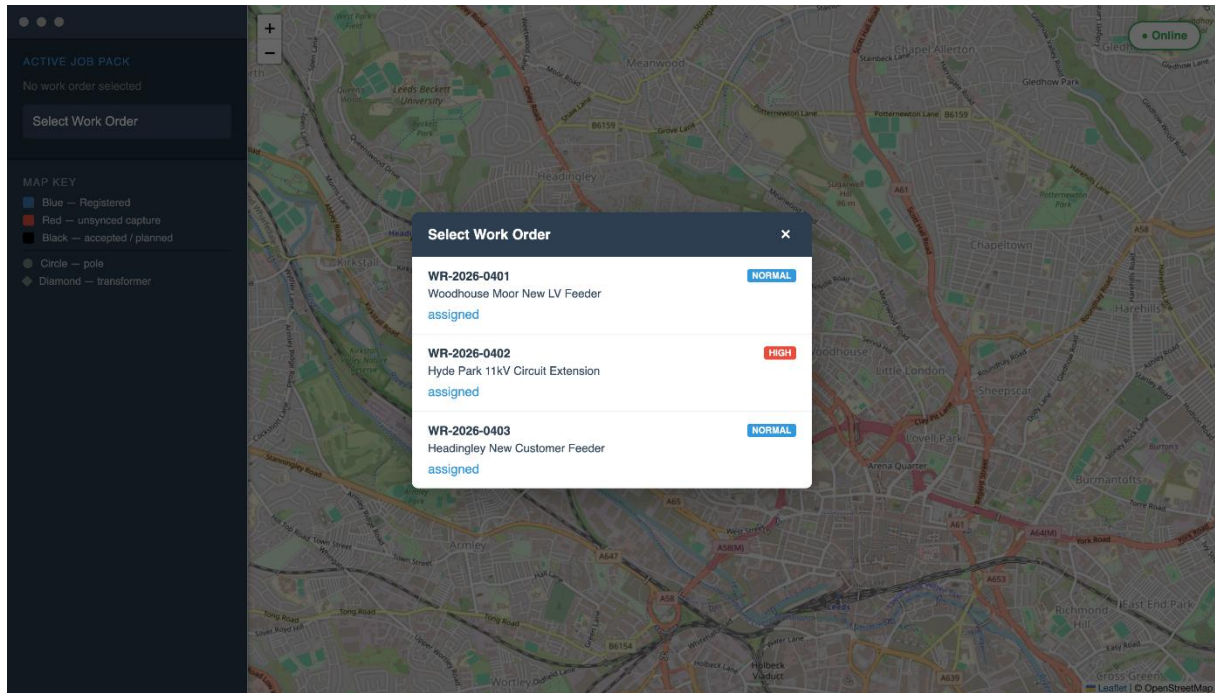


Figure C.1.1: Select work order view.

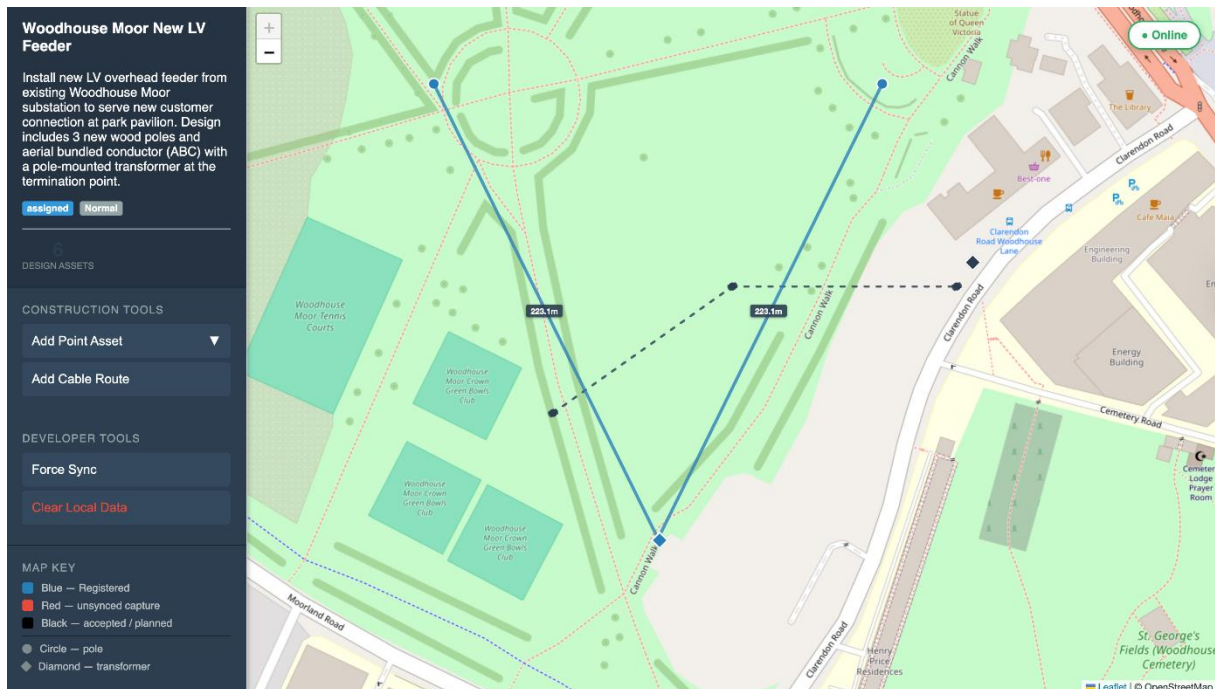


Figure C.1.2: Open work order with a connection to the server (online mode).

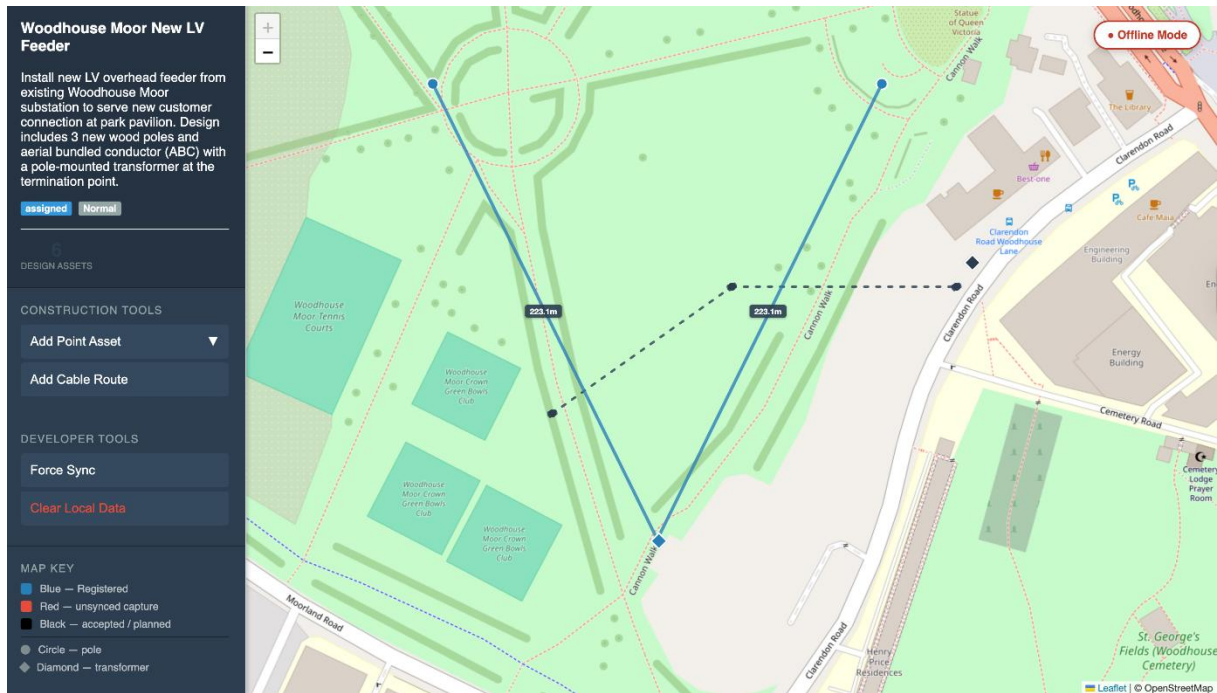


Figure C.1.3: Open work order without a connection to the server (offline mode).

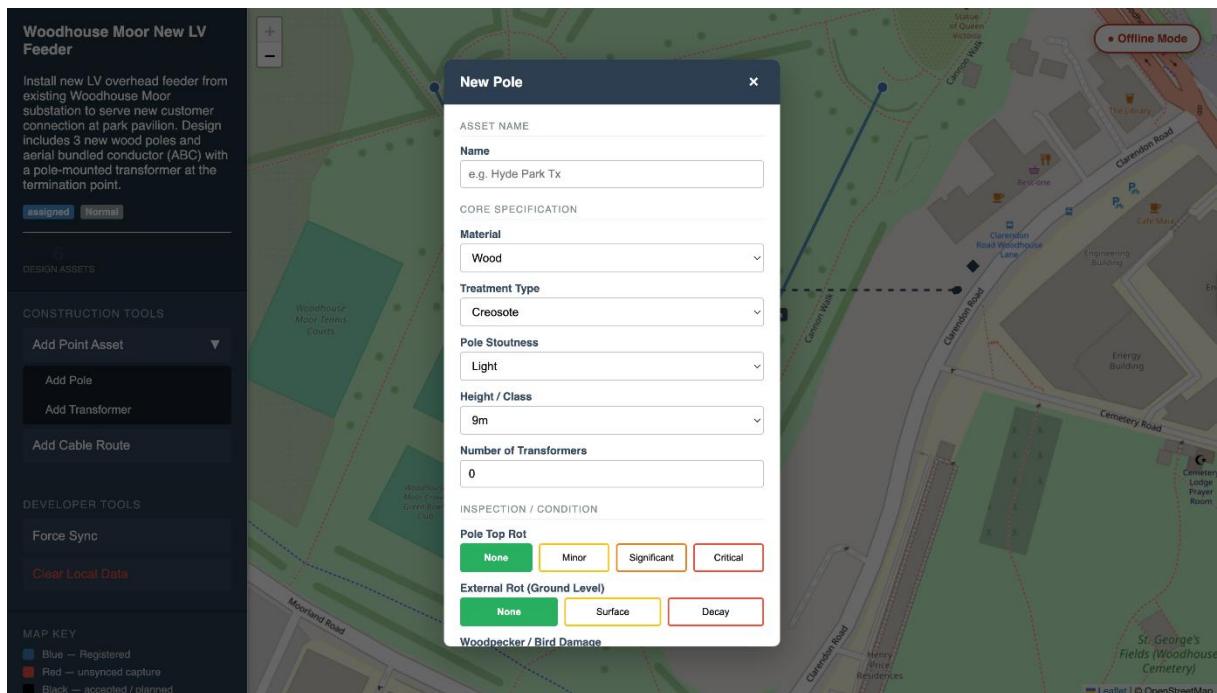


Figure C.1.4: Example of an asset data capture form.

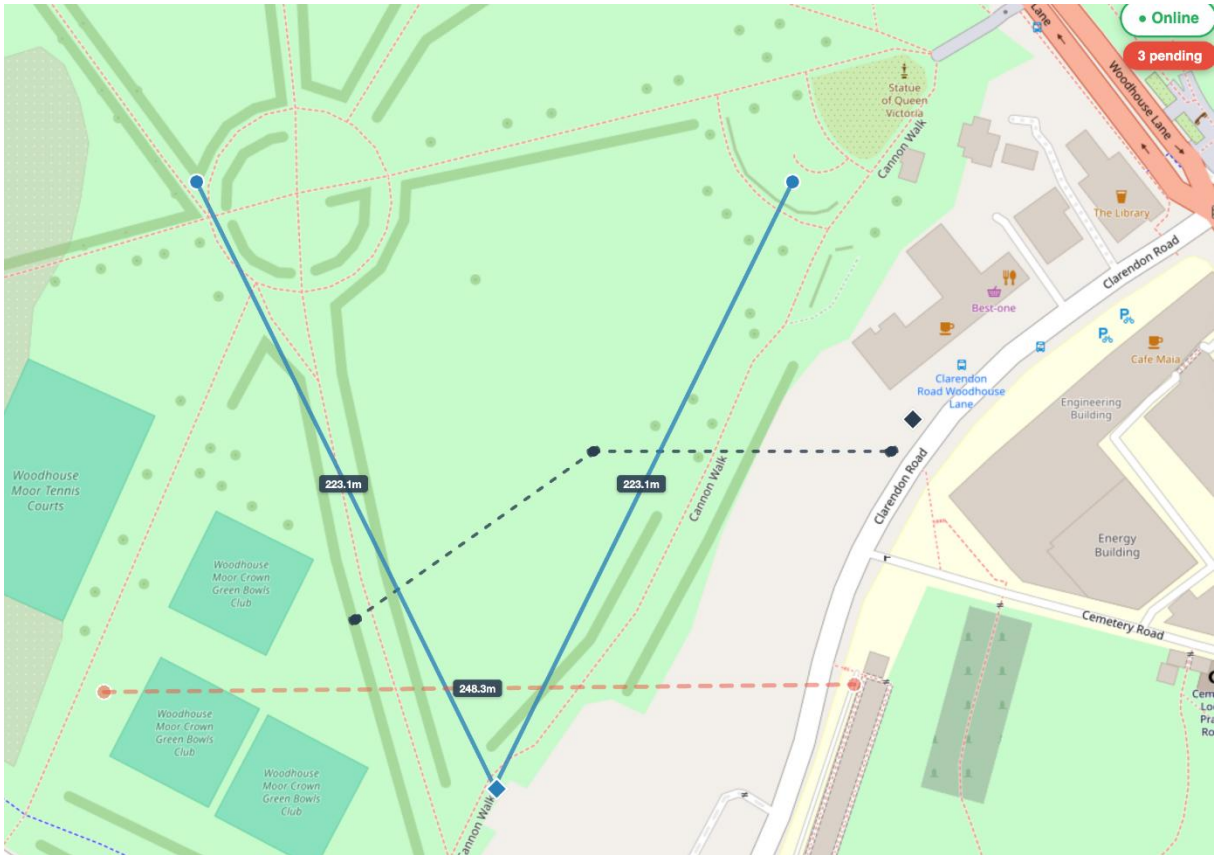


Figure C.1.5: Example of what a redlined (as-built/as-laid) design looks like before a sync is triggered. In this case, two poles are connected by a single cable.



Figure C.1.6: Redlined asset has been synced with the backend, and the assets are now persistent in the register, changing the red line to a blue line.

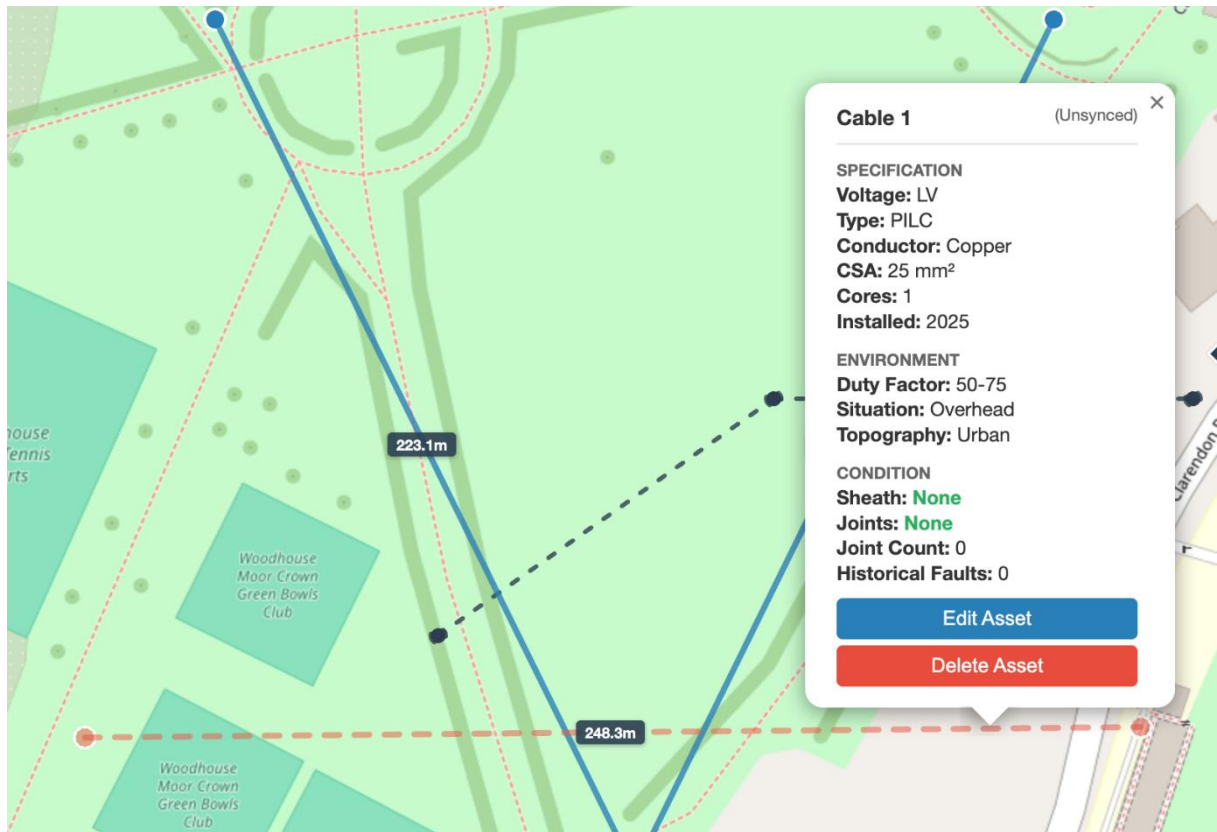


Figure C.1.7: Post drawn asset data capture form and asset options view.

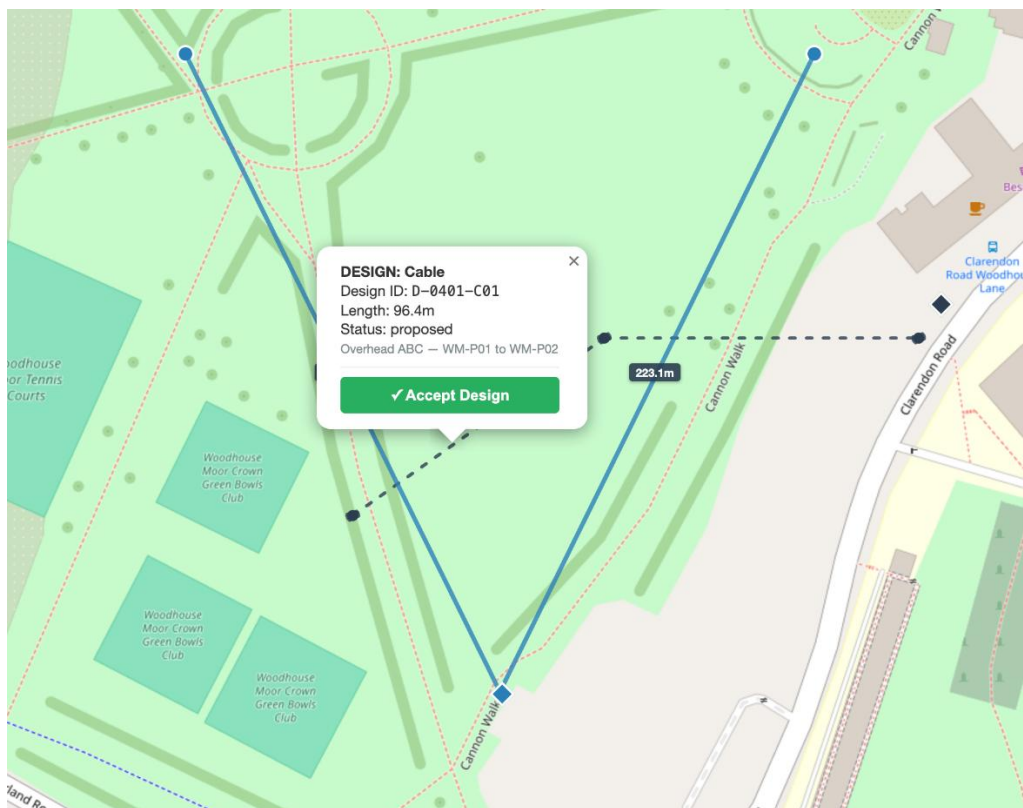


Figure C.1.8: View upon selecting a blacklined design.

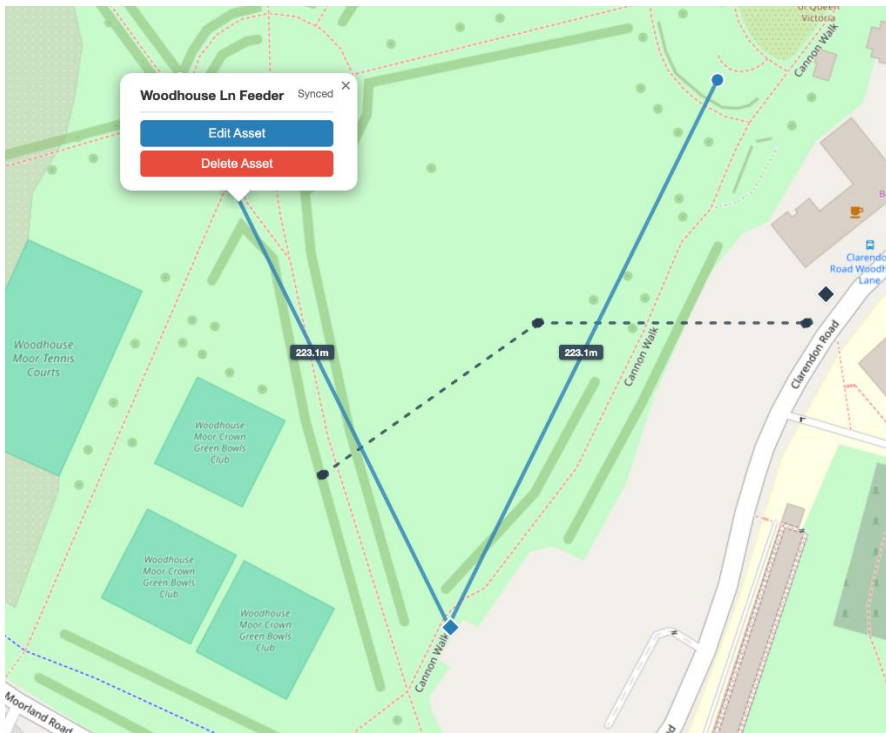


Figure C.1.9: View when selecting an existing network asset.

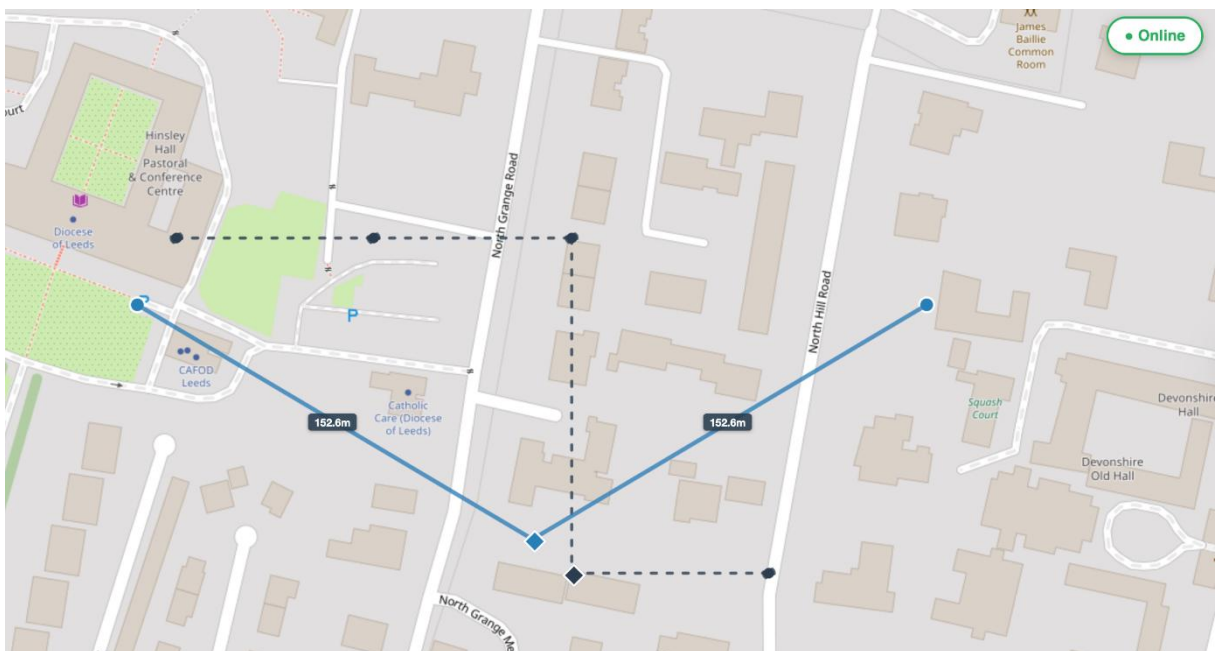


Figure C.1.10: Clean blacklined design prior to acceptance.



Figure C.1.11: View showing how, once a blackline is accepted, the UI updates the representation from a dashed line to a solid line (pre-sync).

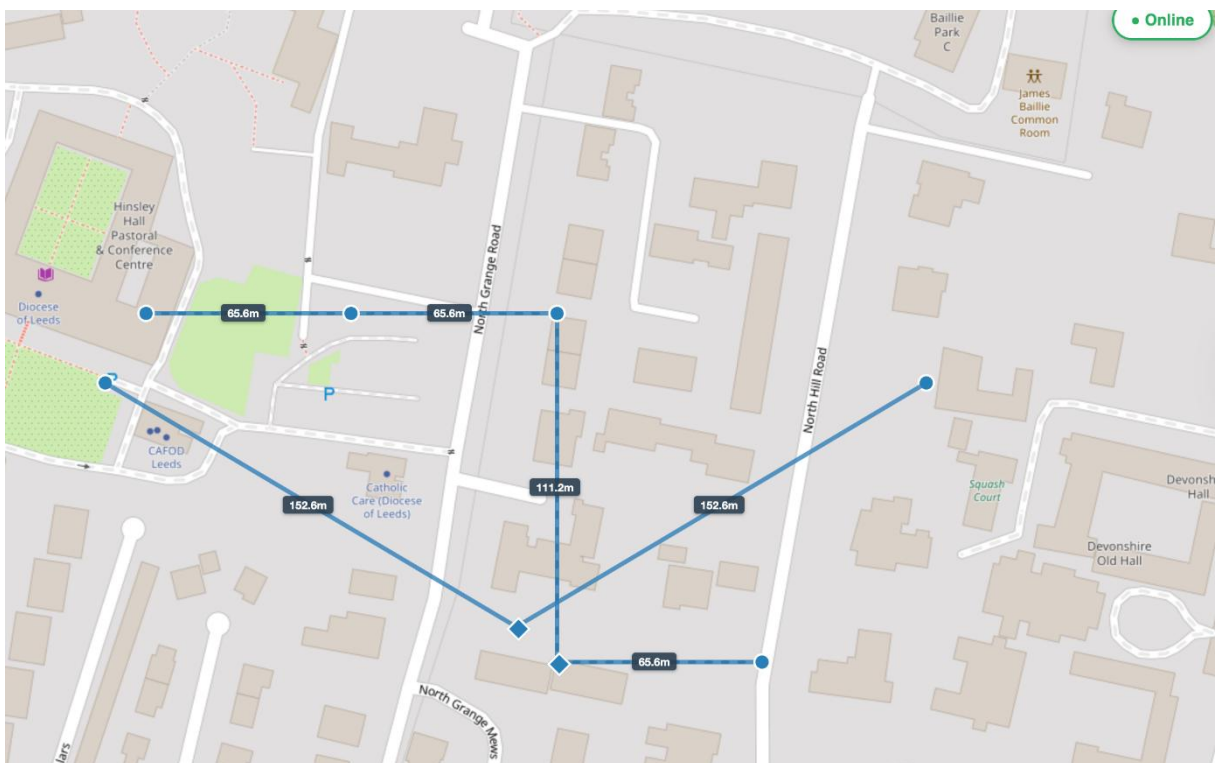


Figure C.1.12: Once blacklines are synced to the asset register, they are now confirmed network assets and switch to a blue line.

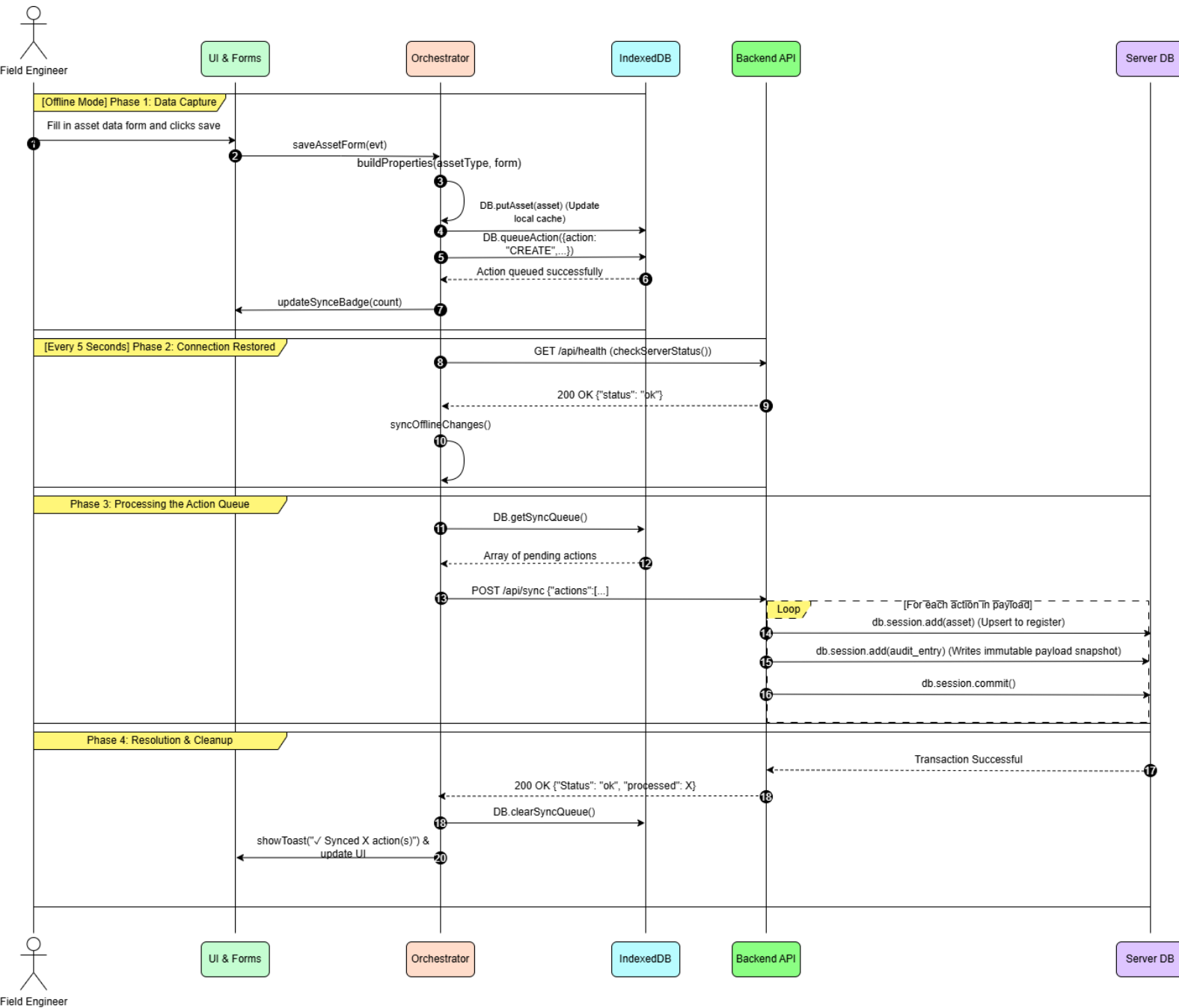


Figure C.1.13 – UML sequence diagram describing the action queue logic

C.2 Supplementary Figures: Browser Metrics

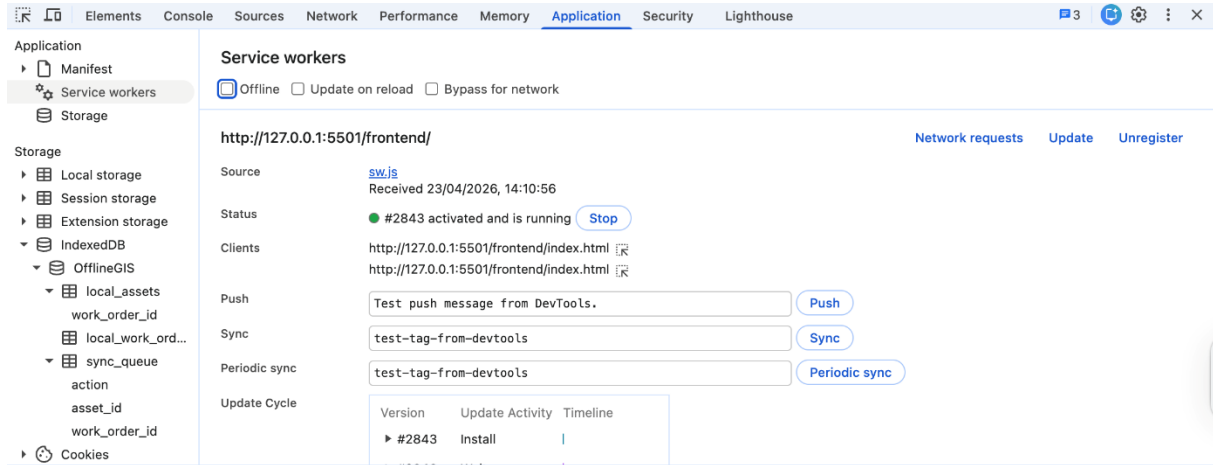


Figure C.2.1: Service Worker registration page showing that the application has successfully registered a service worker with this browser.

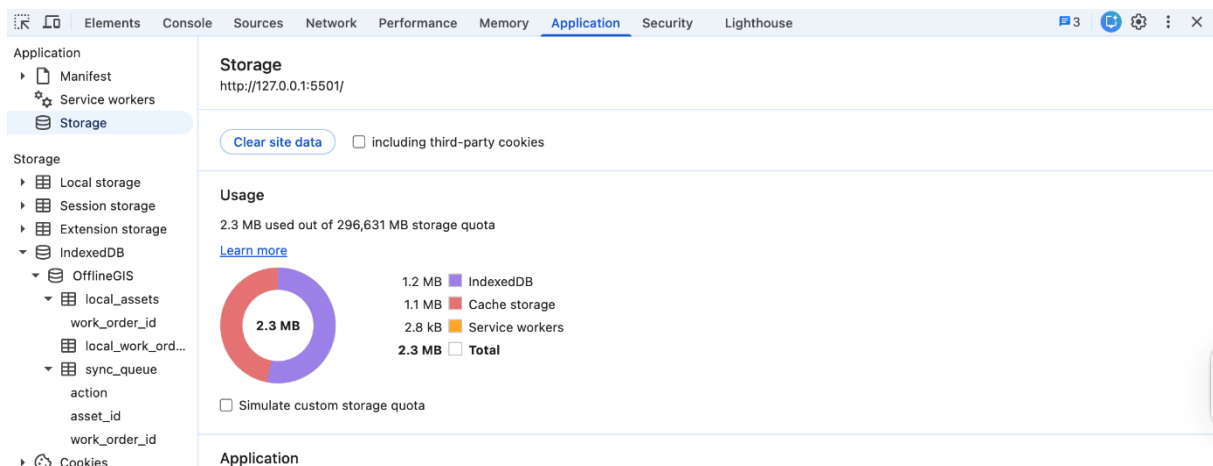


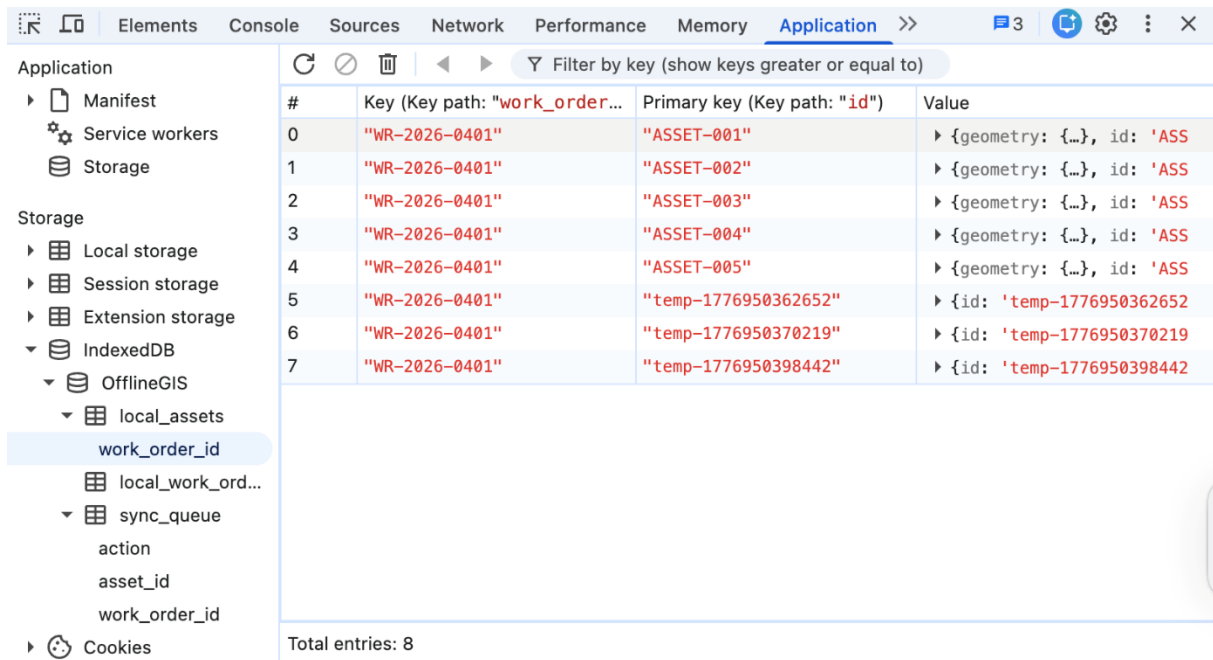
Figure C.2.2: Storage usage of the PWA components totalling 2.3MB.

The screenshot shows the Chrome DevTools Application panel with the 'Recorder' tab selected. The left sidebar shows the application structure. The main panel displays a table of recorded actions. The table has columns for '#', 'Key (Key path: "action")', 'Primary key (Key path: "id")', and 'Value'. It shows three 'CREATE' actions for different assets: a 'Pole' (id: 168), another 'Pole' (id: 169), and a 'Cable' (id: 170). Each entry includes detailed JSON data for the action, asset_id, and geometry.

#	Key (Key path: "action")	Primary key (Key path: "id")	Value
0	"CREATE"	168	{action: 'CREATE', asset_id: 'temp-1776950362652', work_order_id: 'WR-2026-041', asset_id: "temp-1776950362652", asset_type: "Pole", geometry: {coordinates: (2) [-1.5619683265686037, 53.80928330038988], type: "Point", id: 168}, properties: {name: 'Pole 1', assetType: 'Pole', status: 'As-Built', created_timestamp: "2026-04-23T13:19:22.653Z", work_order_id: "WR-2026-0401"}}
1	"CREATE"	169	{action: 'CREATE', asset_id: 'temp-1776950370219', work_order_id: 'WR-2026-041'}
2	"CREATE"	170	{action: 'CREATE', asset_id: 'temp-1776950398442', work_order_id: 'WR-2026-041', asset_id: "temp-1776950398442", asset_type: "Cable", geometry: {coordinates: (2) [Array(2), Array(2)], type: "LineString", id: 170}}

Total entries: 3 | Key generator value: 171

Figure C.2.3: IndexedDB action queue showing the serialisation of actions and their GeoJSON strings.



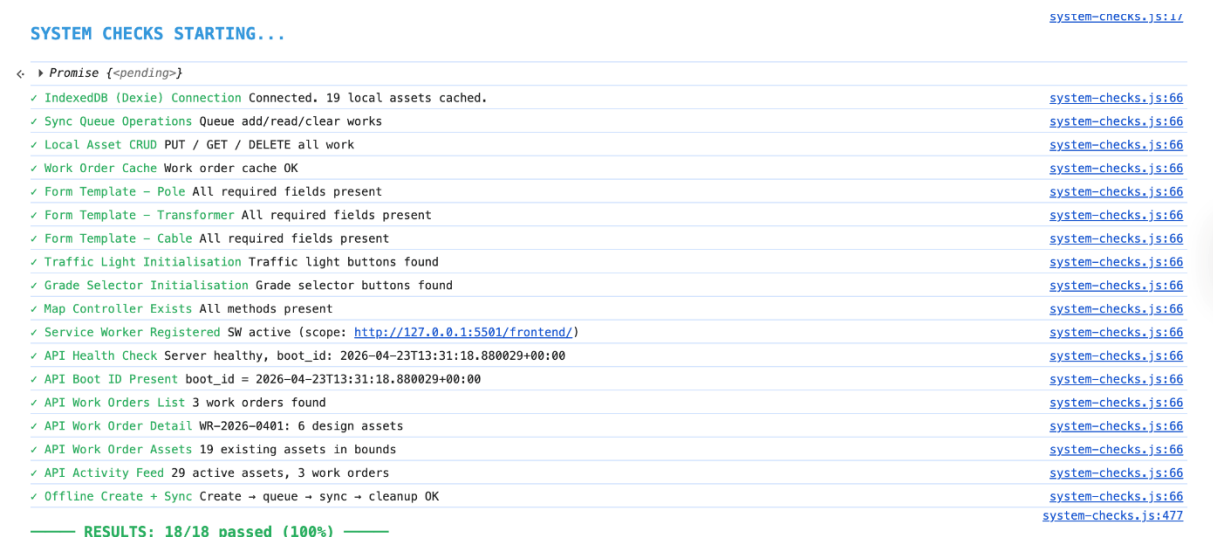
The screenshot shows the Chrome DevTools Application tab. On the left, the 'IndexedDB' database is expanded, showing a collection named 'work_order_id'. The main pane displays a table of 8 entries. The table has four columns: '#', 'Key (Key path: "work_order...")', 'Primary key (Key path: "id")', and 'Value'. The values are GeoJSON strings representing different assets and temporary IDs.

#	Key (Key path: "work_order...")	Primary key (Key path: "id")	Value
0	"WR-2026-0401"	"ASSET-001"	{geometry: {...}, id: 'ASS
1	"WR-2026-0401"	"ASSET-002"	{geometry: {...}, id: 'ASS
2	"WR-2026-0401"	"ASSET-003"	{geometry: {...}, id: 'ASS
3	"WR-2026-0401"	"ASSET-004"	{geometry: {...}, id: 'ASS
4	"WR-2026-0401"	"ASSET-005"	{geometry: {...}, id: 'ASS
5	"WR-2026-0401"	"temp-1776950362652"	{id: 'temp-1776950362652
6	"WR-2026-0401"	"temp-1776950370219"	{id: 'temp-1776950370219
7	"WR-2026-0401"	"temp-1776950398442"	{id: 'temp-1776950398442

Total entries: 8

Figure C.2.4: Local assets, both cached and added, in an example session.

C.3 Test Results



The screenshot shows the results of system checks in a browser. The title is 'SYSTEM CHECKS STARTING...'. Below it, a list of 18 checks is shown, all of which passed. The checks include IndexedDB connection, Sync Queue operations, Local Asset CRUD, Work Order Cache, Form Template validation, Traffic Light Initialisation, Grade Selector Initialisation, Map Controller existence, Service Worker registration, API Health Check, API Boot ID, API Work Orders List, API Work Order Detail, API Work Order Assets, API Activity Feed, and Offline Create + Sync Create. The final result is 'RESULTS: 18/18 passed (100%)'.

Check	Result
IndexedDB (Dexie) Connection Connected. 19 local assets cached.	Passed
Sync Queue Operations Queue add/read/clear works	Passed
Local Asset CRUD PUT / GET / DELETE all work	Passed
Work Order Cache Work order cache OK	Passed
Form Template - Pole All required fields present	Passed
Form Template - Transformer All required fields present	Passed
Form Template - Cable All required fields present	Passed
Traffic Light Initialisation Traffic light buttons found	Passed
Grade Selector Initialisation Grade selector buttons found	Passed
Map Controller Exists All methods present	Passed
Service Worker Registered SW active (scope: http://127.0.0.1:5501/frontend/)	Passed
API Health Check Server healthy, boot_id: 2026-04-23T13:31:18.880029+00:00	Passed
API Boot ID Present boot_id = 2026-04-23T13:31:18.880029+00:00	Passed
API Work Orders List 3 work orders found	Passed
API Work Order Detail WR-2026-0401: 6 design assets	Passed
API Work Order Assets 19 existing assets in bounds	Passed
API Activity Feed 29 active assets, 3 work orders	Passed
Offline Create + Sync Create → queue → sync → cleanup OK	Passed

RESULTS: 18/18 passed (100%)

Figure C.3.1: Browser-based system checks passing with 100% tests passed.

```
plugins: dash=3.3.0, Faker=23.0.0, anyio=4.3.0, timeout=2.4.0
collected 33 items

backend/test_backend.py::TestHealth::test_health_get PASSED [ 3%]
backend/test_backend.py::TestHealth::test_health_head PASSED [ 6%]
backend/test_backend.py::TestHealth::test_boot_id_is_consistent PASSED [ 9%]
backend/test_backend.py::TestWorkOrders::test_list_returns_array PASSED [ 12%]
backend/test_backend.py::TestWorkOrders::test_list_excludes_design_assets PASSED [ 15%]
backend/test_backend.py::TestWorkOrders::test_list_contains_expected_fields PASSED [ 18%]
backend/test_backend.py::TestWorkOrders::test_detail_includes_design_assets PASSED [ 21%]
backend/test_backend.py::TestWorkOrders::test_detail_not_found PASSED [ 24%]
backend/test_backend.py::TestWorkOrders::test_assets_endpoint PASSED [ 27%]
backend/test_backend.py::TestWorkOrders::test_assets_not_found_wo PASSED [ 30%]
backend/test_backend.py::TestSyncCreate::test_create_single PASSED [ 33%]
backend/test_backend.py::TestSyncCreate::test_created_asset_appears_in_register PASSED [ 36%]
backend/test_backend.py::TestSyncCreate::test_create_idempotent PASSED [ 39%]
backend/test_backend.py::TestSyncCreate::test_create_cable PASSED [ 42%]
backend/test_backend.py::TestSyncCreate::test_create_transformer PASSED [ 45%]
backend/test_backend.py::TestSyncUpdate::test_update_existing PASSED [ 48%]
backend/test_backend.py::TestSyncUpdate::test_update_nonexistent_creates PASSED [ 51%]
backend/test_backend.py::TestSyncDelete::test_soft_delete PASSED [ 54%]
backend/test_backend.py::TestSyncDelete::test_deleted_asset_excluded_from_register PASSED [ 57%]
backend/test_backend.py::TestSyncDelete::test_delete_nonexistent_no_error PASSED [ 60%]
backend/test_backend.py::TestSyncAccept::test_accept_design PASSED [ 63%]
backend/test_backend.py::TestSyncAccept::test_accept_marks_design_accepted PASSED [ 66%]
backend/test_backend.py::TestSyncAccept::test_accept_creates_audit_entry PASSED [ 69%]
backend/test_backend.py::TestSyncEdgeCases::test_batch_multiple PASSED [ 72%]
backend/test_backend.py::TestSyncEdgeCases::test_empty_payload PASSED [ 75%]
backend/test_backend.py::TestSyncEdgeCases::test_invalid_action_skipped PASSED [ 78%]
backend/test_backend.py::TestSyncEdgeCases::test_missing_asset_id_skipped PASSED [ 81%]
backend/test_backend.py::TestSyncEdgeCases::test_no_body PASSED [ 84%]
backend/test_backend.py::TestAuditLog::test_audit_records_created PASSED [ 87%]
backend/test_backend.py::TestAuditLog::test_audit_contains_expected_fields PASSED [ 90%]
backend/test_backend.py::TestActivity::test_activity_returns_stats PASSED [ 93%]
backend/test_backend.py::TestActivity::test_activity_returns_recent PASSED [ 96%]
backend/test_backend.py::TestAdmin::test_admin_page_loads PASSED [100%]

===== 33 passed in 0.28s =====
```

Figure C.3.2: Terminal output after running test_backend.py, showing the system passes with 100% pass rate.

C.4 Technical and Repository Supplementary Information

```
1. | backend/
2. |   | app.py           # Flask API server (8 routes, seed helpers)
3. |   | models.py        # SQLAlchemy models: WorkOrder, Asset, AuditLog
4. |   | database.geojson  # Seed data – 15 existing assets across 3 areas
5. |   | work_orders.json  # Seed data – 3 work orders with design assets
6. |   | requirements.txt  # Python dependencies
7. |   | test_backend.py   # 33 pytest API endpoint tests
8. |
9. | frontend/
10. |   | index.html       # PWA shell – Dexie CDN, SW registration, app view
11. |   | styles.css       # All CSS including sync badge and form styles
12. |   | app.js           # Main orchestrator (provision → capture → sync)
13. |   | db.js            # Dexie.js IndexedDB wrapper (3 stores)
14. |   | api.js           # REST API communication layer
15. |   | state.js         # Shared AppConfig and AppState objects
16. |   | map.js           # Leaflet map setup, layer rendering, draw tools
17. |   | ui.js            # UI helpers – panels, modals, sidebar controls
18. |   | forms.js         # CNAIM-aligned form templates (pole, transformer, cable)
19. |   | sw.js            # Service Worker – static cache + tile cache
20. |   | manifest.json    # PWA manifest for standalone mode
21. |
22. | tests/
23. |   | system-checks.js # 18 in-browser integration tests (run via console)
```

Figure C.4.1: Repository structure.

```
1. {
2.   "actions": [
3.     {
4.       "action": "CREATE",
5.       "asset_id": "WR-2025-9901-1748302841649",
6.       "work_order_id": "WR-2025-9901",
7.       "geometry": { "type": "Point", "coordinates": [-1.558, 53.81] },
8.       "properties": { "asset_type": "Pole", "material": "Wood" }
9.     },
10.    {
11.      "action": "UPDATE",
12.      "asset_id": "ASSET-003",
13.      "geometry": { "type": "Point", "coordinates": [-1.559, 53.811] },
14.      "properties": { "condition": "Good" }
15.    },
16.    {
17.      "action": "DELETE",
18.      "asset_id": "WR-2025-9901-1748302799000"
19.    },
20.    {
21.      "action": "ACCEPT",
22.      "asset_id": "design-asset-001",
23.      "work_order_id": "WR-2026-0401",
24.      "asset_type": "Pole",
25.      "geometry": { "type": "Point", "coordinates": [-1.558, 53.81] },
26.      "properties": { "status": "As-Built" }
27.    }
28.  ]
29. }
```

Figure C.4.2: Sync payload format.

Method	Path	Description
GET	/api/health	Returns { "status": "ok", "boot_id": "..." }. Used by the frontend to detect connectivity and server restarts
GET	/api/workorders	Returns all work orders as a JSON array (id, reference, name, description, status, priority, assigned_to, assigned_date, due_date, bounds, asset_count)
GET	/api/workorders/<id>	Returns a single work order including its design_assets GeoJSON
GET	/api/workorders/<id>/assets	Returns existing as-built assets filtered to the work order's geographic bounding box
POST	/api/sync	Accepts { "actions": [...] }. Processes each action (CREATE / UPDATE / DELETE / ACCEPT) sequentially and writes audit log entries
GET	/api/audit	Returns the audit log, most recent first (limited to 100 entries)
GET	/api/activity	Returns database statistics (active assets, decommissioned, work orders, audit entries) and the 50 most recent audit entries
GET	/admin	Serves a live activity dashboard that polls /api/activity every two seconds

Table C.4.1: API endpoints and their associated method, path and description.

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \arctan2(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

where ϕ = latitude in radians, λ = longitude in radians, and $R = 6,371,000$ m.

Equation C.4.1: Haversine Formula used to calculate cable length, written in JavaScript.